multicampus

알고리즘 기초

본 자료는 멀티캠퍼스 교육 서비스의 무단전재 및 배포를 금합니다.
Copyright 2021. emi all rights reserved.

CONTENTS

- 정렬
- 검색
- 순회
- 문자표현, 패턴매칭
- 스택과 큐
- 재귀
- 리스트
- 집합
- 순열
- 그래프

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

복 습

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

if

조건문과 반복문

- 주어진 조건에 따라 다른 문장을 선택할 수 있도록 하는 것을 말합니다.

제어문의 사용 방법

```
if ( boolean 혹은 Condition ) {
    ...
}
```

- Condition은 비교문을 말합니다. JAVA 에서 사용하는 비교 연산자들이 사용됩니다.
  때에 따라 논리 연산자도 함께 사용됩니다.

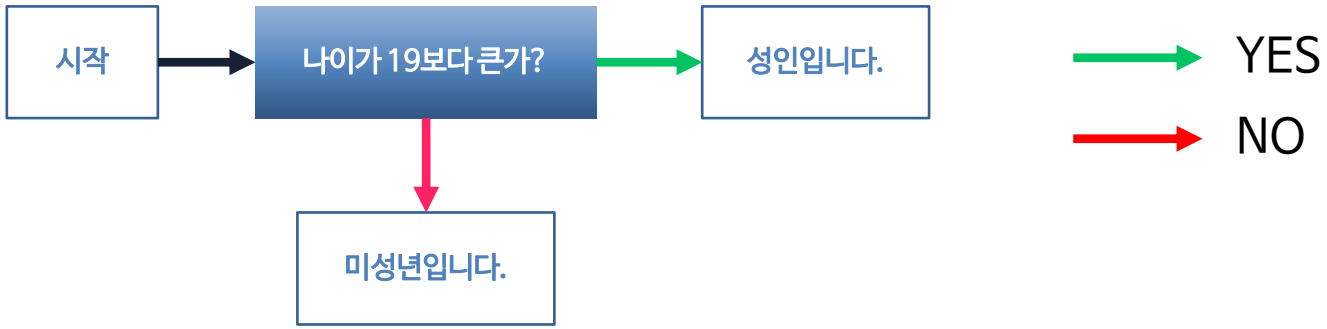
  비교 연산자는 크다, 작다, 같다, 다르다, 크거나 같다, 작거나 같다 등으로 구분됩니다.
  논리 연산자는 그리고, 또는, 부정 등으로 구분됩니다.

- 위 두개의 연산자는 결과값이 boolean 입니다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

if ~ else ~

- if 는 else와 함께 사용하면 비교연산의 결과에 따라 다른 처리를 할 수 있습니다.



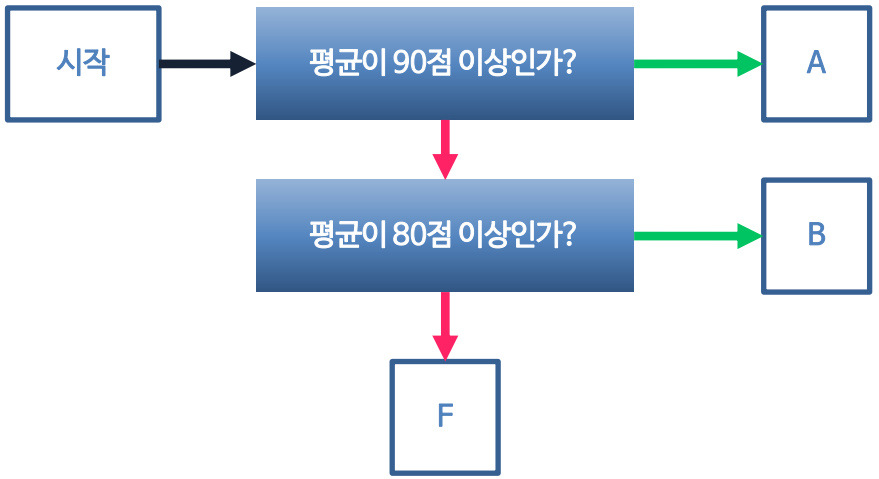
if ~ else 를 이용한 참/거짓 처리

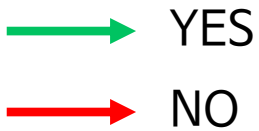
```
int age = 40;
if ( age 〉 19 ) {
   System.out.println( “성인입니다.” );
}
else {
   System.out.println( “미성년입니다.” );
}
```

본 교재는 copyright 202409 금합니다. rights reserved.

if ~ else if ~ else

- if 는 else if, else와 함께 사용하면 아주 디테일한 제어문을 만들어낼 수 있습니다.





if ~ else if ~ else를 이용한 학점 계산

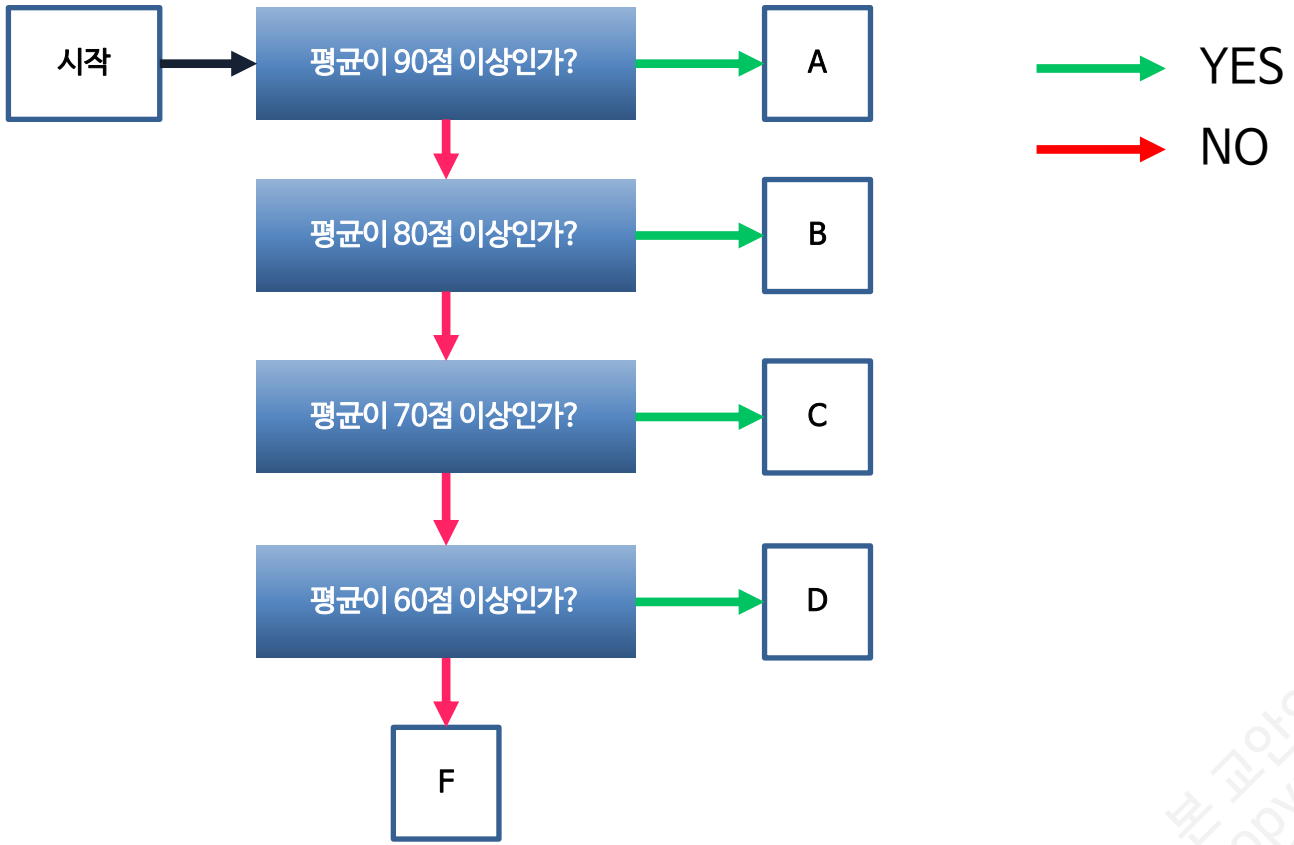
```
double average = 95.5;
String grade = "";
if ( average >= 90 ) {
   grade = "A";
}
else if ( average >= 80 ) {
   grade = "B";
}
else {
   grade = "F";
}
```

if ~ else if ~ else

- **학점의 기준**

  A. 90점 이상, B. 80점 이상, C. 70점 이상, D. 60점 이상, F. 60점 미만

  위의 기준으로 실습 예제 14-1 번의 평균으로 학점을 구해봅니다.



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

if ~ else if ~ else

- if ~ else if ~ else문은 하나의 조건을 만족하면 다른 조건은 더 이상 비교하지 않습니다!

if ~ else if ~ else를 이용한 학점 계산

```
double average = 95.5;
String grade = “”;
if ( average 〉= 90 ) {
   grade = “A”;
}
else if ( average 〉= 80 ) {
   grade = “B”;
}
else if ( average 〉= 70 ) {
   grade = “C”;
}
else if ( average 〉= 60 ) {
   grade = “D”;
}
else {
   grade = “F”;
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

if

조건문과 반복문

▪ if-else if 문과 if-if 문의 차이

if ~ else if ~ else

```
double average = 95.5;
String grade = "";
if ( average >= 90 ) {
   grade = "A";
}
else if ( average >= 80 ) {
   grade = "B";
}
else if ( average >= 70 ) {
   grade = "C";
}
else if ( average >= 60 ) {
   grade = "D";
}
else {
   grade = "F";
}
```

if ~ if

```
double average = 95.5;
String grade = "";
if ( average >= 90 ) {
   grade = "A";
}
if ( average >= 80 ) {
   grade = "B";
}
if ( average >= 70 ) {
   grade = "C";
}
if ( average >= 60 ) {
   grade = "D";
}
else {
   grade = "F";
}
```

본 교재는 copyright 202409 멀티캠퍼스 All rights reserved.

if

조건문과 반복문

- if 제어문은 비교연산자와 논리연산자를 사용할 수 있습니다.

논리연산자 문법

```
boolean result = (값 비교연산자 값) 논리연산자 (값 비교연산자 값);
```

예를 들어, A학점의 조건을 아래와 같이 정한다고 한다면,

1. 평균이 90점을 넘어야 한다.
2. 지각/결석이 0회여야 한다.

우리는 논리연산자를 사용할 수 있습니다.

A 학점의 조건 - 1 (boolean 사용)

```
double average = 91.6;
int dayoff = 0 ;
boolean result = ( average >= 90 ) && ( dayoff == 0 )

if ( result ) {
    ...
}
```

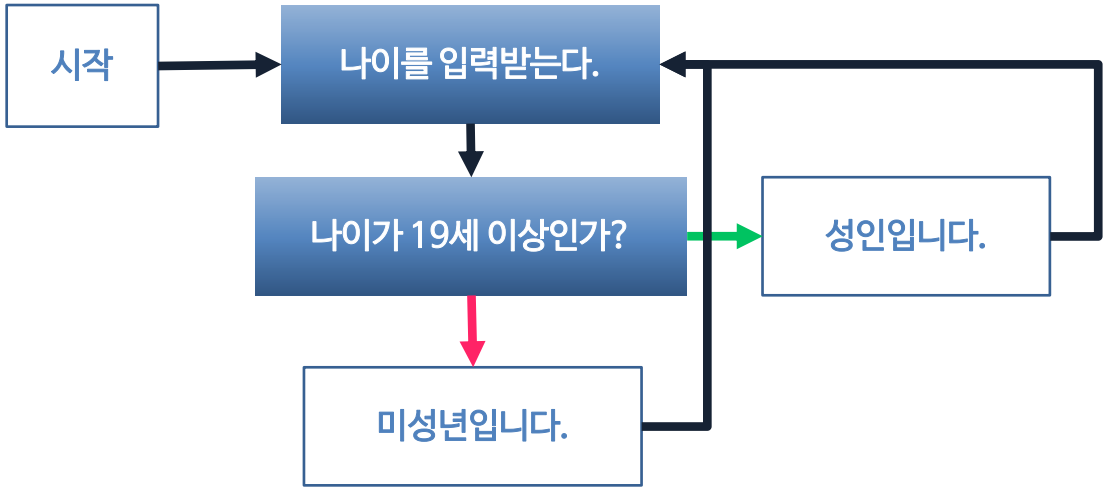
A 학점의 조건 - 2 (Condition 사용)

```
double average = 91.6;
int dayoff = 0 ;

if ( ( average >= 90 ) && ( dayoff == 0 ) ) {
    ...
}
```

본 교재는 copyright 202409 배포를 금합니다. All rights reserved.

반복문

- **반복문은 특정 코드 블럭을 지정된 횟수 또는 무한히 반복을 시키려 할 때 사용합니다.**
  특히 배열처럼 연속된 데이터들을 조회(탐색)할 때 주로 사용됩니다.
  나이를 입력 받고 성인/미성년을 구분하는 코드를 무한히 반복하려 한다면 아래처럼 작성할 수 있습니다.



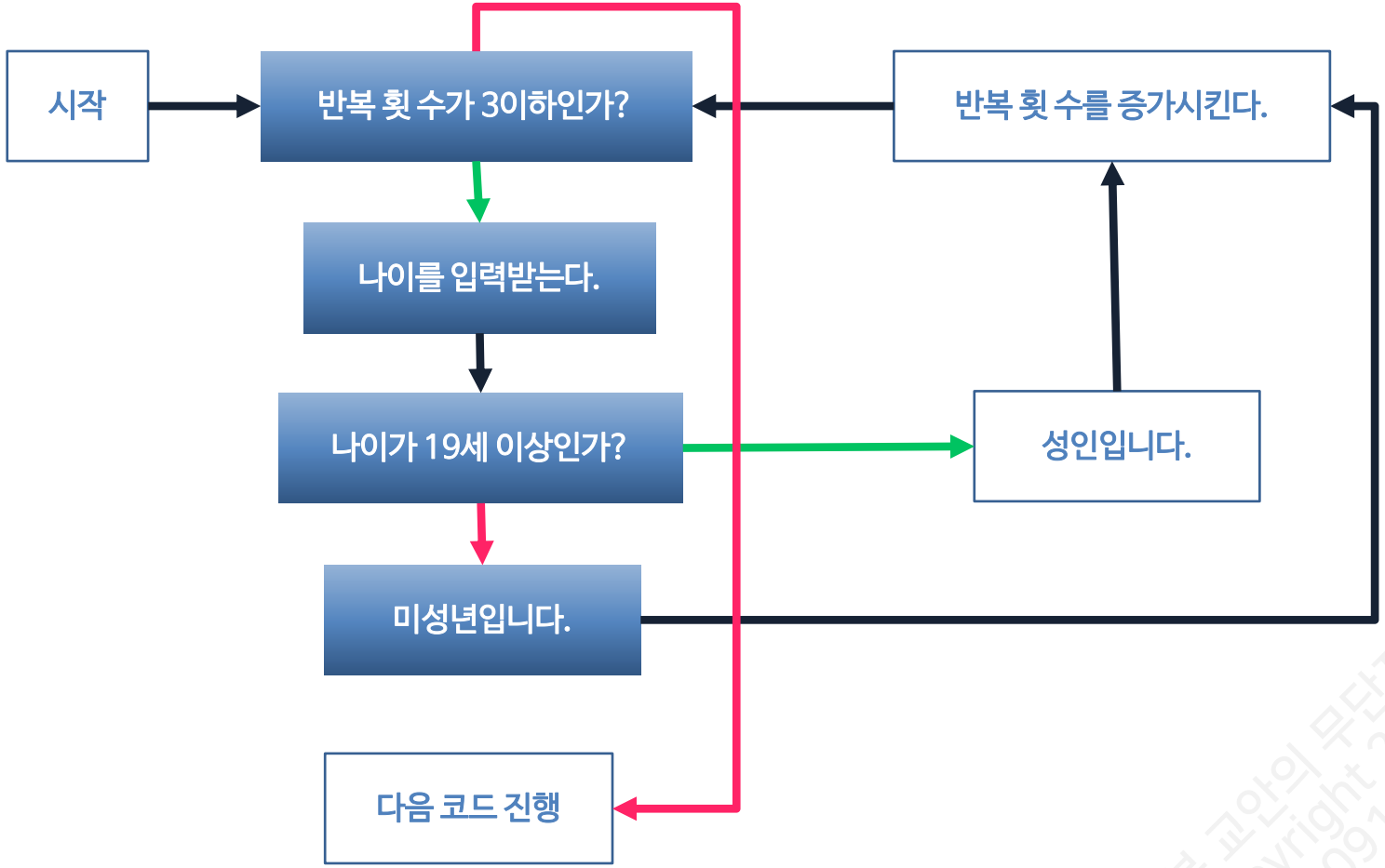
- **하지만, 무한히 반복하는 것은 위험하므로 일반적인 반복문을 보도록 하겠습니다.**

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

반복문

조건문과 반복문

- 아래 다이어그램은 총 3번만 반복하도록 설계되었습니다.



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

while

조건문과 반복문

일반 코드

```
int loopCount = 1;

if ( loopCount <= 3 ) {

   int age = 30;
   if ( age >= 19 ) {
      System.out.println( “성인입니다.” );
   }
   else {
      System.out.println( “미성년입니다.” );
   }

   loopCount = loopCount + 1;
}
```

반복문 (반복 체크 문장을 while 로 변경)

```
int loopCount = 1;

while ( loopCount <= 3 ) {

   int age = 30;
   if ( age >= 19 ) {
      System.out.println( “성인입니다.” );
   }
   else {
      System.out.println( “미성년입니다.” );
   }

   loopCount = loopCount + 1;
}
```

while 의 문법 (boolean or Condition 이 true 일 때만 반복합니다.)

```
while ( boolean or Condition ) {
   ...
}
```

위 코드는 이전 페이지의 다이어그램을 코드화 시킨 것 입니다. **while 은 JAVA 가 지원하는 가장 원시적인 반복문입니다.**

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240909

do ~ while

- while문은 조건을 먼저 검사하기 때문에 조건식에 맞지 않으면 반복이 한 번도 일어나지 않습니다.
  무조건 한 번의 수행이 이루어 진 후에 조건식을 검사하기 위해 do ~ while문을 사용합니다.

반복문

```
int num = 1;
int sum = 0;

do {
  sum += num;
  num++;
} while ( num <= 5 );

System.out.println( "1부터 5까지의 합은 " + sum + " 입니다." );
```

do ~ while 의 문법

```
do {
  수행문1;
  ...
} while ( boolean or Condition );
  수행문2;
  ...
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

for

조건문과 반복문

- 반복문 중에서 가장 많이 사용하는 반복문이 for문입니다.

  while문 보다는 필요한 조건식들이 있어야 해서 조금 복잡합니다.

반복문 (반복 체크 문장을 while 로 변경)

```
int loopCount = 1;

while ( loopCount <= 3 ) {

    int age = 30;
    if ( age >= 19 ) {
        System.out.println( "성인입니다." );
    }
    else {
        System.out.println( "미성년입니다." );
    }

    loopCount = loopCount + 1;
}
```

일반 코드

```
for ( int i = 1; i <= 3; i++ ) {

    int age = 30;
    if ( age >= 19 ) {
        System.out.println( "성인입니다." );
    }
    else {
        System.out.println( "미성년입니다." );
    }
}
```

for 의 문법 (i <= 반복횟수의 결과가 true 일 때만 반복합니다.)

```
for ( int i = 0; i <= 반복횟수; i++ ) {
    ...
}
```

무단전재 및 재배포를 금합니다.
Copyright 2021. emily all rights reserved.
20240919

for

- for문을 가장 많이 사용하는 이유는 반복 횟수가 가능하기 때문입니다.
  while문에 비해 for문은 한 줄에 초기화, 조건비교, 증감식까지 쓸 수 있고 가독성도 좋습니다.

for 의 문법 (i <= 반복횟수의 결과가 true 일 때만 반복합니다.)

```
for ( int i = 0; i <= 5; i++ ) {
  ...
}
```

1. i를 0으로 초기화 합니다.
2. 조건식 i<=5를 검사했을 때 i는 0이므로 참입니다.
3. 조건식이 참이기 때문에 조건문을 수행합니다. 증감식 i++를 수행하여 i는 1이 됩니다.

...

* 조건식 i<=5를 검사했을 때 i값은 6이므로 거짓입니다.
  For문이 종료됩니다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

중첩 for

- 반복문 안에 또 다른 반복문을 중첩해서 사용하는 예가 종종 있습니다.
  외부 for문과 내부 for문이 어떤 순서로 실행되는지 잘 이해해야 합니다.
  for문 외에 다른 반복문도 중첩해서 사용이 가능합니다.

for 의 문법 (i <= 반복횟수의 결과가 true 일 때만 반복합니다.)

```
for ( int i = 0; i <= 5; i++ ) {
  for ( int j = 0; j <= 5; j++ ) {
    ...
  }
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

Array

배열

Array(배열)

- JAVA 는 숫자, 문자, 문자열, 참/거짓 정보를 변수라는 공간에 저장할 수 있도록 지원합니다.
  그리고 한 변수는 반드시 하나의 값만 저장할 수 있습니다.
  점수라는 공통적인 성격의 변수가 필요할 때가 있습니다.
  이 경우 필요한 변수의 개수를 일일이 선언하는 것보다 **"점수" 라는 변수를 선언해 여러 개의 값을 저장**할 수도 있습니다.
- 자료를 순차적으로 저장할 수 있는 이러한 구조를 Array(배열)이라 합니다.

배열의 선언 방법

```
데이터_타입[] 배열_변수_명 = new 데이터_타입[변수의 개수];
```

정수형 배열의 선언

```
int[] scores = new int[4];
```

C Style의 정수형 배열의 선언

```
int scores[] = new int[4];
```

- JAVA 는 JAVA Style 과 C Style 의 배열 선언 방법 모두를 지원합니다.
  그러나 C Style 은 잘 사용하지 않습니다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

Array

배열

Array 선언

정수형 배열의 선언

```
int[] scores = new int[4];
```

정수형 배열에 값 할당

```
scores[0] = 100;
scores[1] = 90;
scores[2] = 97;
scores[3] = 93;
```

- 배열 변수는 각 배열 값을 0번부터 시작해 선언한 숫자만큼 할당 시킬 수 있습니다.
  여기에 사용된 0, 1, 2, 3 값들은 **인덱스** 라고 부릅니다. 배열의 순서는 0부터 시작합니다.
  인덱스는 배열 선언 개수만큼 사용할 수 있습니다.
- 주의할 점은 scores 배열 변수는 int 타입으로 선언되었으므로 int 에 맞는 타입의 값만 넣을 수 있습니다.
  (정수만 넣을 수 있습니다.)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

Array의 사용

- 배열에 값을 할당하기 위해 인덱스를 사용한 것 처럼 배열의 값을 알기 위해서 인덱스를 사용합니다.

배열의 사용

```
int total = scores[0] + scores[1] + scores[2] + scores[3];
```

- 배열은 위 코드처럼 여러 개의 변수를 선언하지 않고도 많은 값을 할당할 수 있도록 해줍니다.
- JAVA에서 배열은 사용 빈도가 높진 않습니다.

  하지만, 종종 쓰일 경우가 있고 그 개념을 알아야 다른 것을 쉽게 받아들이기 때문에 **완벽하게 이해해야 합니다.**
- 한번 정의된 배열의 크기는 줄이거나 늘리거나 할 수 없습니다. 아래와 같은 코드는 오류를 발생합니다.

배열의 잘못된 사용 방법

```
int[] scores = new int[4];
scores[0] = 100;
scores[1] = 90;
scores[2] = 97;
scores[3] = 93;

scores = new int[6];
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

배열과 반복문

- **반복문과 함께 사용되는 배열**
  - 반복문은 배열을 탐색하기 위해 존재합니다.

    배열의 특성과 증감에 따른 반복을 표현하는 데 적합한 for문의 특성 때문에 자주 사용하게 되는 것입니다.
  - 같은 데이터 타입 그리고 같은 성격의 데이터를 모아 놓은 배열을 처음부터 끝까지(지정 인덱스 부터 지정 인덱스 까지) 탐색하고자 할 때 사용됩니다.

    배열 순서를 나타내는 색인은 항상 0부터 시작하기 때문에 배열의 전체 요소 개수가 n개 일 때, 요소의 위치는 n-1로 표현이 가능합니다.

    반복문을 사용하지 않는 배열 탐색은 복잡하고 번거롭지만, 반복문을 사용하면,인덱스가 아무리 많더라도 아주 간단하게 탐색 코드를 작성할 수 있습니다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

배열과 반복문

multicampus

반복문을 사용하지 않는 배열 탐색

```
int[] scores = new int[4];
scores[0] = 97;
scores[1] = 90;
scores[2] = 91;
scores[3] = 98;

System.out.println( "영어 : " + scores[0] );
System.out.println( "수학 : " + scores[1] );
System.out.println( "과학 : " + scores[2] );
System.out.println( "프로그래밍 : " + scores[3] );

int sum = scores[0] + scores[1] + scores[2] + scores[3];
```

반복문을 사용하는 배열 탐색

```
int[] scores = new int[4];
scores[0] = 97;
scores[1] = 90;
scores[2] = 91;
scores[3] = 98;

int sum = 0;

for ( int i = 0; i < scores.length; i++ ) {
   System.out.println( "scores[" + i + "] : " + scores[i] );
   sum = sum + scores[i];
}
```

본 교안의 무단전재 및 ... 다.
copyright 2021. emily ... reserved.
20240919

multicampus

알고리즘

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

알고리즘

- **알고리즘**

  주어진 문제를 해결하기 위한 단계적인 절차

  주로 컴퓨터용어로 쓰이며, 컴퓨터가 어떤 일을 수행하기 위한 단계적 방법을 말한다.

  어떠한 문제를 해결하기 위한 절차

- **알고리즘의 조건**
  - 입력 : 모호하지 않고 잘 정의된 입력 값
  - 출력 : 명확히 정의된 1개 이상의 출력 값
  - 명확성 : 각 명령어의 의미가 모호하지 않고 명확해야 한다
  - 유한성 : 한정된 수의 단계 후에는 반드시 종료되어야 한다
  - 언어 독립성 : 프로그래밍 언어와 상관없이 독립적이다
  - 유효성 : 명령어들은 현재 실행 가능한 연산이어야 한다

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘

- **좋은 알고리즘이란**

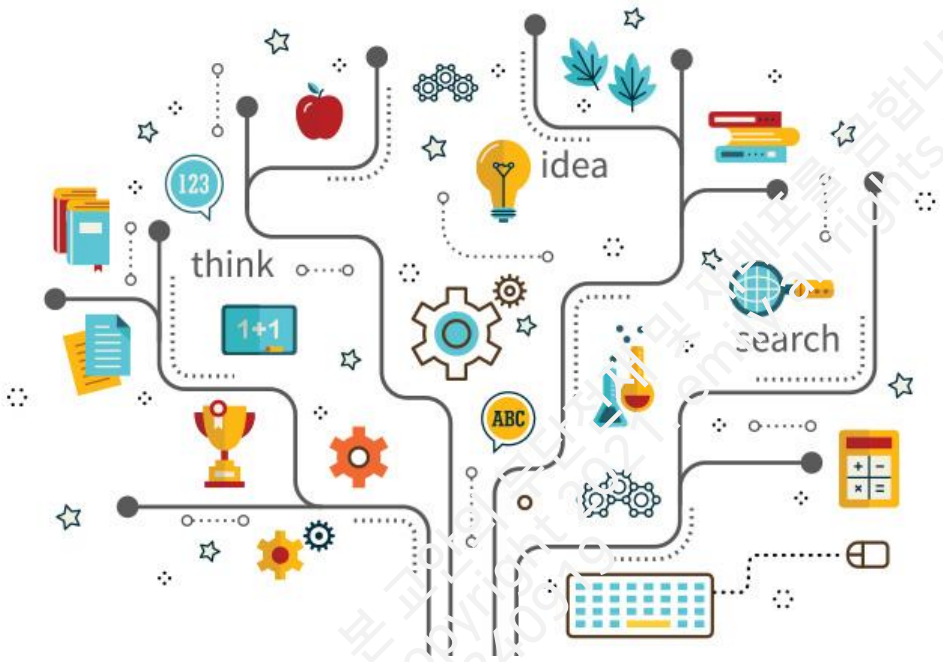
① 정확성 : 얼마나 정확하게 동작하는가

② 작업량 : 얼마나 적은 연산으로 원하는 결과를 얻어내는가

③ 메모리 사용량 : 얼마나 적은 메모리를 사용하는가

④ 단순성 : 얼마나 단순한가

⑤ 최적성 : 더 이상 개선할 여지없이 최적화되었는가



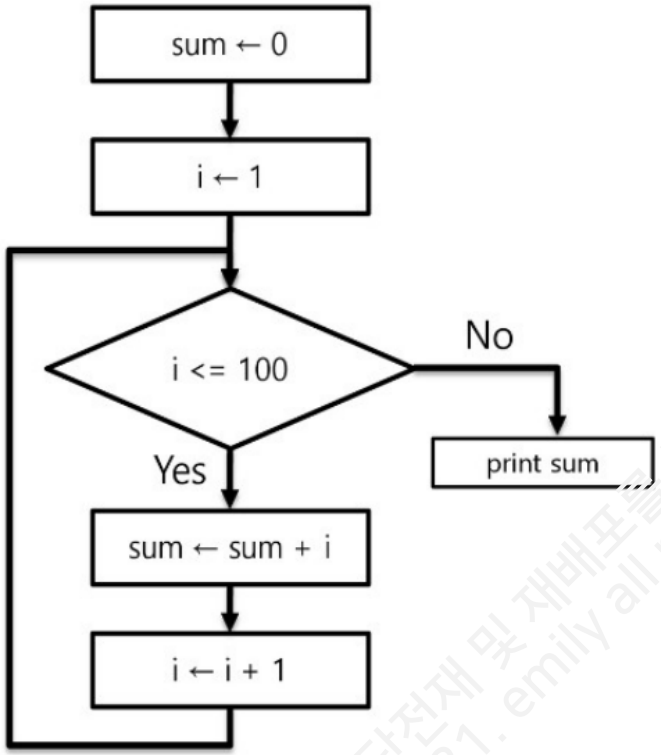
본 교재의 저작권은 멀티캠퍼스에 있음을 밝힙니다.
copyright 2021 multicampus. All rights reserved.

알고리즘의 표현

- **컴퓨터 분야에서 알고리즘을 표현 하는 방법**
  - 영어나 한국어와 같은 자연어를 사용하는 방법
  - 흐름도(순서도, flowchart)로 표현하는 방법
  - 특정한 프로그래밍 언어(java, python, c 등)로 표현하는 방법
  - 유사코드(Pesudo code)로 표현하는 방법

```
CalcSum( n ) {
    sum ← 0
    for ( i ← 1; i <= n; i ← i + 1) {
        sum ← sum + i;
    }
    return sum;
}
```

Pesudocode 예



순서도 예

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- **주어진 문제를 해결하기 위해 여러 개의 다양한 알고리즘이 있을 때, 성능을 비교하기 위해 다양한 기준을 사용할 수 있다.**
  - 연산량

    알고리즘이 수행해야 하는 작업(연산)의 양. 알고리즘의 **시간 효율성**을 나타냄.
  - 메모리 사용량

    필요한 작업을 하는데 사용하는 기억 공간의 양. 알고리즘의 **공간 효율성**을 나타냄.

    예전에 비해 컴퓨터 메모리 효율이 좋아짐에 따라 공간 효율성의 중요성이 지금은 조금 떨어진 경향이 있다.

- **가장 좋은 알고리즘은 연산량과 메모리 사용량이 모두 작은 것이겠지만 둘을 동시에 만족시키기는 쉽지 않다.**

  따라서, 둘 중 하나를 골라야 한다면 보통은 시간 효율성을 선택한다.
  - 시간 효율성 측정을 위해 실행 시간을 직접 측정해 볼 수는 있지만, 여러 약점이 존재

    알고리즘을 반드시 구현해야 한다든가, 반드시 같은 조건의 하드웨어를 사용해야 한다든가, 소프트웨어 환경도 동일해야 한다든가, 입력된 데이터에 따라 다른 결과가 나올 수도 있기 때문에 실험되지 않은 입력에 대해 실행 시간을 주장할 수 없다든가

본 교안의 무단 복제 및 재배포를 금합니다.
copyright 2022 Kemily all rights reserved.
20240919

알고리즘 성능

- **복잡도 분석(complexity analysis)**
  - 시간을 측정하지 않고 알고리즘의 성능을 평가하는 방법
  - 구현하지 않고도 알고리즘의 시간 복잡도와 공간 복잡도를 구해 성능을 비교하는 방법

- **1부터 100까지 합을 구하는 문제**

알고리즘 1	알고리즘 2
1 + 2 + 3 + 4 ..... + 100 5,050	$\frac{100\times(1+100)}{2}=5,050$
99번의 연산 (덧셈 99번)	3번의 연산 (덧셈 1번, 곱셈 1번, 나눗셈 1번)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- **알고리즘의 작업량을 표현할 때 시간복잡도로 표현**
- **시간 복잡도(Time Complexity)**
  - 실제 걸리는 시간을 측정
  - 실행되는 명령문의 개수를 계산

알고리즘 1	알고리즘 2

`CalcSum( n ) {`
`  sum ← 0;  // 1번`
`  for ( i ← 1; i <= n; i ← i+1)  // 1+1번`
`  sum ← sum + i; // 1 번`
`  return sum;`
`}`
	`CalcSum( n ) {`
	`  return n*(n+1)/2  //3번`
	`}`

1 + n * 3 = 3n + 1	3번의 연산

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- **복잡도의 점근적 표기**
  - 알고리즘의 복잡도는 더 간단한 형태로 단순화시켜 사용
  - 여러 항을 갖는 복잡도 함수를 최고차 항만을 계수 없이 취해 단순하게 표현하는 방법을 사용
  - 정확히 몇 번의 연산이 필요한가 가 아니라 연산량이 얼마나 빨리 증가하는가 만을 나타냄(증가속도를 표현)

알고리즘 1	알고리즘 2

`CalcSum( n ) {`
`  sum ← 0;  // 1번`
`  for ( i ← 1; i <= n; i ← i+1)  // 1+1번`
`  sum ← sum + i; // 1 번`
`  return sum;`
`}`
	`CalcSum( n ) {`
	`  return n*(n+1)/2  //3번`
	`}`

1 + n * 3 = 3n + 1	3번의 연산

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- **시간 복잡도 ≒ 빅-오(0) 표기법**
  - 빅-오 표기 법 (Big-Oh Notation)
  - **시간 복잡도 함수 중에서 가장 큰 영향력을 주는 n에 대한 항만을 표시**
  - 계수(Coefficient)는 생략하여 표시
  - 예를 들어 n개의 데이터를 입력 받아 저장한 후 각 데이터에 1씩 증가시킨 후 각 데이터를 화면에 출력하는 알고리즘의 시간복잡도는 어떻게 되나? O(n)

$$O(3n+2) = \underbrace{O(3n)}_{\text{최고차항(3n)만 선택}} = \underbrace{O(n)}_{\text{계수 3제거}}$$

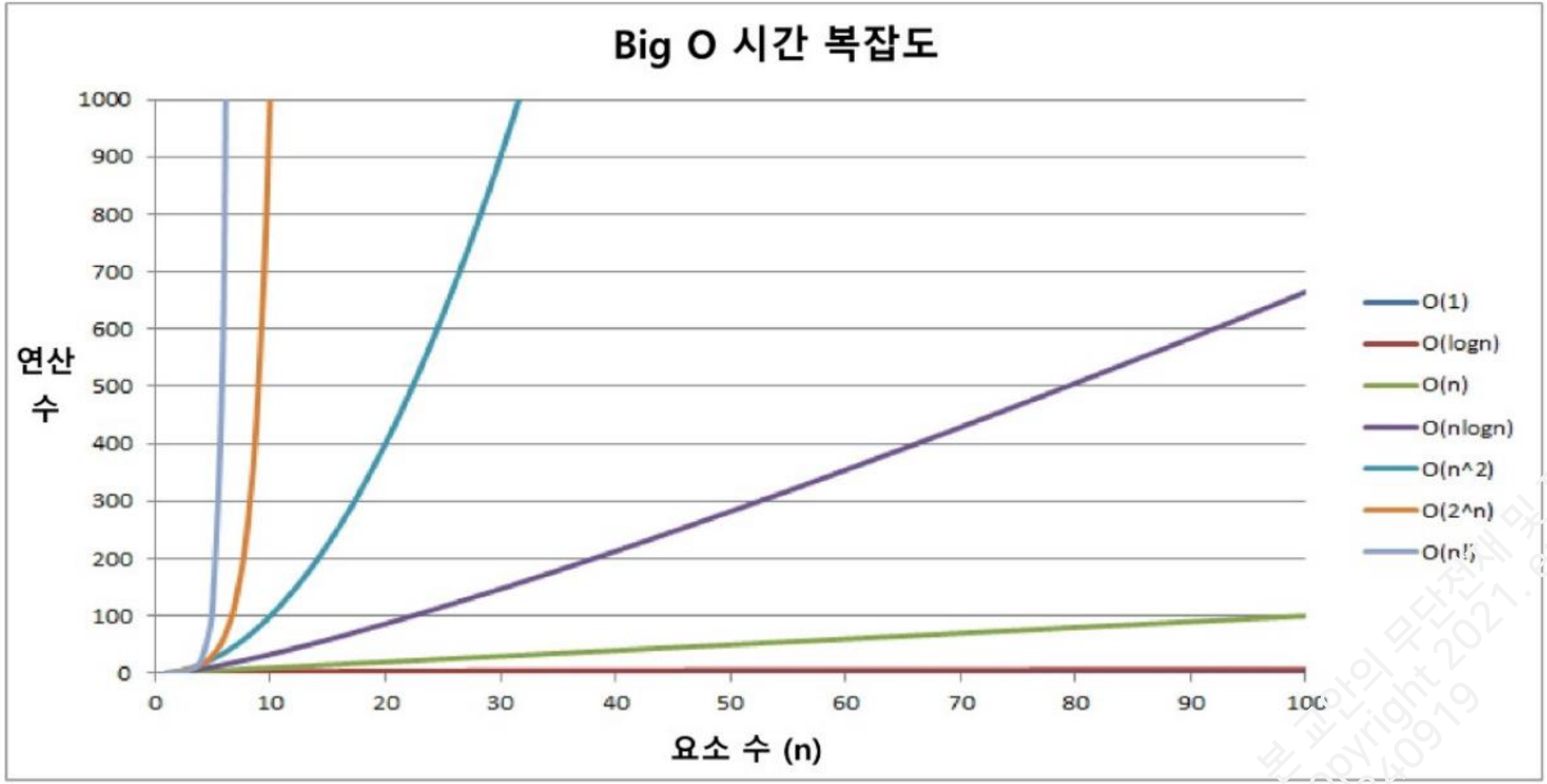
$$O(2n^2 + 10n + 100) = O(n^2)$$

$$O(4) = O(1)$$

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- 요소 수가 증가함에 따라 각기 다른 시간 복잡도의 알고리즘은 아래와 같은 연산 수를 보인다.



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

알고리즘 성능

- 요소 수가 증가함에 따라 각기 다른 시간 복잡도의 알고리즘은 아래와 같은 연산 수를 보인다.

$1\mu s = 10^{-6}$초
$1ms = 10^{-3}$초

	N	$\log N$	N	$N \log N$	N^2	N^3	2^N
	10	0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	1 μs
	20	0.004 μs	0.02 μs	0.086 μs	0.4 μs	8 μs	1 ms
	30	0.005 μs	0.03 μs	0.147 μs	0.9 μs	27 μs	1 s
	40	0.005 μs	0.04 μs	0.213 μs	1.6 μs	64 μs	18.3 min
	50	0.006 μs	0.05 μs	0.282 μs	2.5 μs	125 μs	13 days
	10^2	0.007 μs	0.10 μs	0.664 μs	10 μs	1 ms	4×10^{13} years
	10^3	0.010 μs	1.00 μs	9.966 μs	1 ms	1 s	
	10^4	0.013 μs	10 μs	130 μs	100 ms	16.7 min	
	10^5	0.017 μs	0.10 ms	1.67 ms	10 s	11.6 days	
	10^6	0.020 μs	1 ms	19.93 ms	16.7 min	31.7 days	
	10^7	0.023 μs	0.01 s	0.23 s	1.16 days	31,709 years	
1 억	10^8	0.027 μs	0.10 s	2.66 s	115.7 days	3.17×10^7 years	
	10^9	0.030 μs	1 s	29.90 s	31.7 years		

? 5천만에 대한 연산을 수행한다면?

본 교안의 무단전재 및 재배포를 금합니다.
Copyright 2021. emily all rights reserved.
2024.09.19.

알고리즘 유형

- **코딩 테스트에서 다루는 알고리즘 유형** – 프로그래밍과 문제 해결 능력을 평가하기 위해 다양한 유형의 문제를 다루는 기술을 포함
  - **정렬 알고리즘:** 배열이나 리스트의 원소를 정렬하는 방법. 퀵 정렬, 병합 정렬, 삽입 정렬, 선택 정렬 등.
  - **탐색 알고리즘:** 주어진 자료구조에서 특정한 값을 찾는 방법. 이진 탐색, 선형 탐색 등.
  - **그래프 알고리즘:** 그래프 형태의 자료구조. 그래프의 경로 탐색, 최단 경로 찾기 등이 포함. DFS(Depth-First Search)와 BFS(Breadth-First Search)가 있음
  - **그리디 알고리즘:** 각 단계에서 최적의 선택을 하는 방식으로 문제를 해결하는 방법. 가장 좋은 선택을 하여 최종 결과를 도출.
  - **동적 프로그래밍:** 큰 문제를 작은 하위 문제로 나누어 해결하고, 그 결과를 저장하며 중복 계산을 피하는 기법. 재귀적 접근과 메모이제이션을 통해 구현.
  - **문자열 처리 알고리즘:** 문자열을 다루는데 필요한 알고리즘. 문자열 매칭, 부분 문자열 찾기, 편집 거리 계산 등.
  - **스택과 큐:** 자료를 저장하고 조작하는 데 사용되는 자료구조. 스택(Stack)과 큐(Queue)의 원리와 활용 방법을 이해해야 함.
  - **백트래킹:** 가능한 모든 경우의 수를 탐색하면서 해를 찾는 알고리즘. 일부 불필요한 경우를 미리 제거하면서 탐색 시간을 줄임.
  - **비트 조작 알고리즘:** 비트 연산을 활용하여 문제를 해결하는 방법. 이진수 표현, 비트 마스크 등.
  - **수학적 알고리즘:** 최대공약수, 최소공배수, 소수 판별 등과 같은 수학적 문제를 다루는 방법.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2024 eduwill all rights reserved.
20240919

SW 문제를 완벽하게 풀기 위한 5단계

1. 지문을 읽는다.
2. 문제를 이해한다.
3. 문제를 손으로 푼다.
4. 푼 걸 코딩한다.
5. 디버깅하고 검증한다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

정 렬(I)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

정렬 알고리즘(sorting algorithm)

- **정렬 - 데이터를 정해진 기준에 따라 특성을 기준으로 나열하는 것.**
  오름차순(ascending, 작은 값부터)과 내림차순(descending, 큰 값 부터)의 순서로 재배열

- **정렬의 종류**
  - 버블 정렬(bubble sort)
  - 선택 정렬(selection sort)
  - 카운팅 정렬(counting sort)
  - 삽입 정렬(insertion sort)
  - 병합 정렬(merge sort)
  - 퀵 정렬(quick sort)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

버블 정렬(Bubble Sort)

- 두 인접한 데이터의 크기를 비교해 정렬하는 방법
  인접한 데이터 간의 swap 연산으로 정렬
- 정렬 과정
  - 인접한 데이터 값을 비교
  - 조건에 부합하면 swap연산을 수행
  - 범위가 끝날 때까지 반복
- 교환하며 자리를 이동하는 모습이 물 위에 올라오는 거품 모양과 같다고 하여 버블 정렬이라 함

- 시간복잡도 - $O(n^2)$

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

버블 정렬(Bubble Sort)

- {42, 32, 24, 68, 15} - 첫 번째

42	32	24	68	15
32	42	24	68	15
32	24	42	68	15
32	24	42	68	15
32	24	42	15	68

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

버블 정렬(Bubble Sort)

- {42, 32, 24, 68, 15} - 두 번째

32	24	42	15	68

24	32	42	15	68

24	32	42	15	68

24	32	15	42	68

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

버블 정렬(Bubble Sort)

- {42, 32, 24, 68, 15} - 세번째

24	32	15	42	68

24	32	15	42	68

24	15	32	42	68

24	15	32	42	68

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

버블 정렬(Bubble Sort)

- {42, 32, 24, 68, 15} - 세번째

24	15	32	42	68

15	24	32	42	68

15	24	32	42	68

- 정렬 종료

15	24	32	42	68

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선택 정렬(Selection Sort)

- 대상 데이터에서 최대나 최소 데이터를 나열된 순으로 찾아가며 선택하는 방법

- 정렬 방법
  - 최소값을 찾고
  - 가장 앞에 있는 데이터와 선택된 데이터를 swap
  - 남은 부분이 없을 때까지 반복

- 시간복잡도 - $O(n^2)$

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선택 정렬(Selection Sort)

multicampus

- {42, 32, 24, 68, 15}

42	32	24	68	15
15	32	24	68	42
15	24	32	68	42
15	24	32	68	42
15	24	32	42	68

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

검 색

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

검색(Search)

- 여러 개의 데이터 안에서 원하는 데이터를 찾아내는 알고리즘
- 탐색키(search key, 자료를 구별하여 인식할 수 있는 키)를 가진 항목을 찾는 것

- 검색 종류
  - 순차 검색(선형 검색, Sequential Search)
  - 이진 검색(Binary Search)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

multicampus

- **일렬(배열)로 되어 있는 자료를 순서대로 검색하는 방법**
  - 가장 간단하고 직관적인 방법
  - 순차구조로 된 자료구조에서 원하는 항목을 찾을 때 유용
  - 단순하여 구현은 쉽지만, 검색 대상의 수가 많은 경우 수행시간이 증가하여 비효율적인 부분이 있음

- **정렬되어 있지 않은 경우 검색 과정**
  - 첫 번째 원소부터 순서대로 검색 대상과 키 값이 같은 원소가 있는지 찾음
  - 키 값이 동일한 원소를 찾으면 그 원소의 인덱스를 반환
  - 자료구조의 마지막에 도달할 때 까지 검색 대상을 찾지 못하면 검색 실패

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

- 정렬되어 있지 않은 경우 {4, 9, 11, 23, 2, 19, 7} - 2를 검색

①	4	9	11	23	2	19	7
②	4	9	11	23	2	19	7
③	4	9	11	23	2	19	7
④	4	9	11	23	2	19	7
⑤	4	9	11	23	2	19	7

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

multicampus

- **정렬되어 있지 않은 경우**
  - 찾고자 하는 원소의 순서에 따라 비교회수가 결정
  - 첫 번째 원소를 찾으면 1번 비교, 두번째 원소를 찾으면 2번 비교
  - 정렬되지 않은 자료에서의 순차 검색의 평균 비교 회수
    - (n+1) / 2

- **시간복잡도 – O(n)**

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

- **정렬되어 있는 경우**
  - 자료가 오름차순으로 정렬되어 있다는 가정하에 검색을 실시
  - 자료를 순차적으로 검색하면서 키 값을 비교, 원소의 키 값이 검색 대상의 키 값보다 크면 찾는 원소가 없다는 것으로 판단하여 검색을 종료

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

multicampus

- 정렬되어 있는 경우 {2, 4, 7, 9, 11,19, 23} - 11를 검색

①	2	4	7	9	11	19	23
②	2	4	7	9	11	19	23
③	2	4	7	9	11	19	23
④	2	4	7	9	11	19	23
⑤	2	4	7	9	11	19	23

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

- 정렬되어 있는 경우 {2, 4, 7, 9, 11,19, 23} - 10를 검색

①	2	4	7	9	11	19	23
②	2	4	7	9	11	19	23
③	2	4	7	9	11	19	23
④	2	4	7	9	11	19	23
⑤	2	4	7	9	11	19	23

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 검색(선형 검색, Sequantial Search)

- **찾고자 하는 원소의 순서에 따라 비교회수가 결정**
  - 정렬이 되어 있으므로, 검색 실패를 반환하는 경우 평균 비교회수가 반으로 줄어듦

- **시간복잡도 - O(n)**

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

이진 검색(Binary Search)

- **정렬되어 있는 데이터에서 원하는 값을 찾는 방법**
  - 대상 데이터의 중앙값과 찾고자 하는 값을 비교해 데이터의 크기를 절반씩 줄이면서 찾음
  - 자료는 정렬된 상태여야 함

- **정렬 방법**
  - 자료 중앙에 있는 원소를 고름
  - 중앙 원소의 값과 찾고자 하는 값을 비교
  - 찾고자 하는 값이 중앙 원소의 값보다 작으면 자료의 왼쪽 데이터에 대해서 새로 검색하고, 크다면 자료의 오른쪽 반에 대해 새로 검색을 수행
  - 찾고자 하는 값을 찾을 때 까지 반복

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

이진 검색(Binary Search)

- {2, 4, 7, 9, 11,19, 23} - 7을 검색

2	4	7	9	11	19	23

중앙

2	4	7	9	11	19	23

검색 범위 검색 제외

7 < 9
=> 왼쪽 검색

2	4	7	9	11	19	23

검색 범위

4 < 7
=> 오른쪽 검색

2	4	7	9	11	19	23

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emmail all rights reserved.
20240919

이진 검색(Binary Search)

- 검색 범위의 시작점과 종료점을 이용하여 검색을 반복 수행
- 자료에 삽입이나 삭제가 발생하게 되는 경우 항상 정렬 상태로 유지하는 추가 작업이 필요

- 시간복잡도 – O(log n)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

완전 검색(Exhaustive Search)

- 모든 경우의 수를 다 체크해서 정답을 찾는 방법
- Brute-force 라고도 부름
- 모든 경우의 수를 만들어 테스트 한 후 최종 해법을 도출
- 경우의 수가 상대적으로 작을 때 유용
- 모든 경우의 수를 테스트하기에 수행하는 속도는 느리지만, 답을 찾아내지 못할 확률이 작음
- 알고리즘이라 하기 보다는 문제를 푸는 방법이라고 이해하는 것이 편함.

  문제를 풀 때, 완전 검색으로 접근하여 해답을 도출한 후, 성능 개선을 위한 다른 알고리즘을 사용하고 해답을 찾는 것이 방법

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

완전 검색(Exhaustive Search)

- **방법**
  - Brute-Force : 반복/조건문을 활용하여 모두 테스트 하는 방법
  - 순열(Premutation) : 서로 다른 n개 중 r개를 선택하는 경우의 수
  - 재귀 호출
  - 비트마스크 : 2진수 표현 기법 활용
  - DFS, BFS 활용

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

완전 검색(Exhaustive Search)

- 순열(Premutation)
  - 서로 다른 것들 중 몇 개를 뽑아 나열하는 것
  - 서로 다른 n개 중 r개를 선택하는 경우의 수 -〉 nPr 로 표현
  - 식

    nPr = n*(n-1)*(n-2)* ... *(n-r+1)

    $${}_nP_r = \frac{n!}{(n-r)!}$$
  - nPn = n! 이라 표기

    n! = n * (n-1) * (n-2) * ··· * 2 * 1

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

완전 검색(Exhaustive Search)

- {1,2,3}을 포함하는 모든 순서에 대한 순열을 생성
  - 단순한 생성을 위해 반복문을 사용하게 되면 {1,2,3} 속성의 수인 3개 만큼의 3중 for문을 만들어야 한다

```
for i1 from 1 to 3 {
        for i2 from 1 to 3 {
                if i2 != i1 {
                        for i3 from 1 to 3 {
                                if i3 != i1 and i3 != i2 {
                                        print(i1, i2, i3)
                                }
                        }
                }

        }
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

2차원 배열

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

2차원 배열

2차원 배열의 선언

- 2차원 이상의 다차원 배열은 차원에 따라 index 를 선언

2차원 배열

```
int [] [] arr = new int [2] [3];
```

열의 길이 – arr [i] .legnth

행의 길이
arr.legnth

1	2	3
4	5	6

```
public class TwoDimension {

  public static void main (String [] args) {
    int [] [] arr = {{1,2,3},{4,5,6}};

    for (int i=0; i<arr.length; i++) {
      for (int j=0; j<arr [i] .length; j++) {
        System.out.println (arr [i] [j] );
      }
      System.out.println ();
    }
  }
}
```

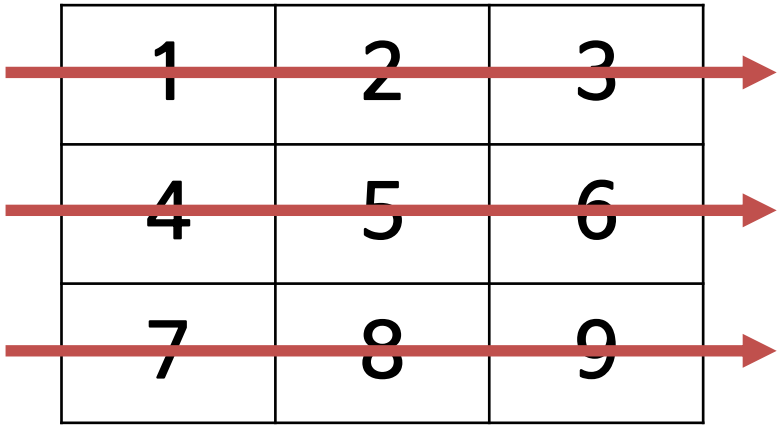
본 교안의 무단
copyright 202
20240919
s reserved.

배열 순회

탐색

- n x m 배열의 n * m 개의 모든 원소를 빠짐없이 조사하는 방법

- 행 우선 순회



행 우선 순회

```
int i;          // 행의 좌표
int j;          // 열의 좌표

int[] [] arr = {{1,2,3},{4,5,6},{7,8,10}};

int N = arr.length;         // 행의 길이
int M = arr[0].length;      // 열의 길이

for(int i = 0; i < N; i++) {
   for(int j = 0; j < M; j++) {
      System.out.printf("%3d", arr[i] [j]);
   }
   System.out.println();
}
```

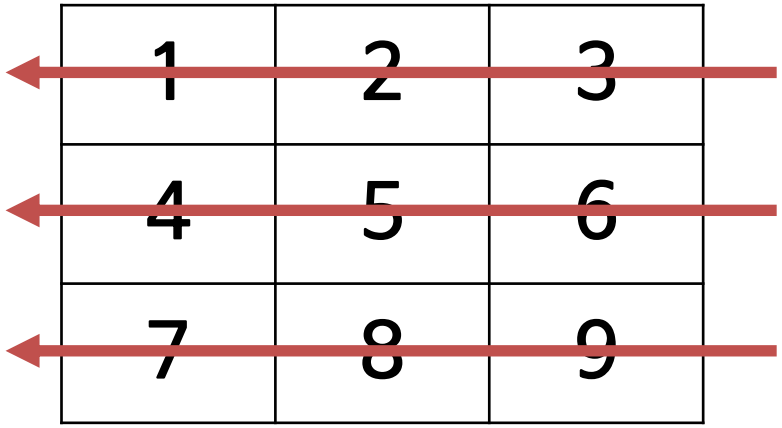
본 교안의 무단 배포를 금지합니다.
copyright 2024 multicampus reserved.
20240919

배열 순회

탐색

- n x m 배열의 n * m 개의 모든 원소를 빠짐없이 조사하는 방법

- 행 역 우선 순회



행 역 우선 순회

```
int i;          // 행의 좌표
int j;          // 열의 좌표

int[] [] arr = {{1,2,3},{4,5,6},{7,8,10}};

int N = arr.length;         // 행의 길이
int M = arr[0].length;      // 열의 길이

for(int i = 0; i < N; i++) {
   for(int j = M-1; j >= 0; j--) {
      System.out.printf("%3d", arr[i] [j]);
   }
  System.out.println();
}
```

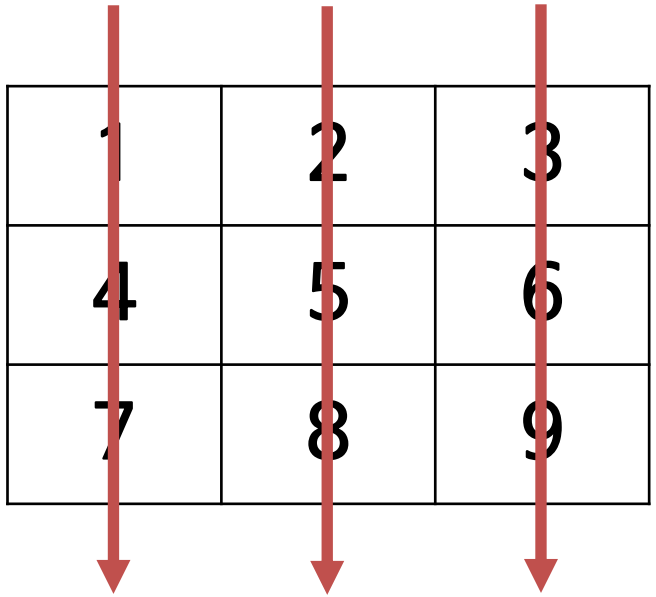
본 교안의 무단 배포 금지. copyright 2024 reserved. 20240919

배열 순회

탐색

- n x m 배열의 n * m 개의 모든 원소를 빠짐없이 조사하는 방법

- 열 우선 순회



열 우선 순회

```
int i;          // 행의 좌표
int j;          // 열의 좌표

int[] [] arr = {{1,2,3},{4,5,6},{7,8,10}};

int N = arr.length;         // 행의 길이
int M = arr[0].length;      // 열의 길이

for(int j = 0; j < M; j++) {
   for(int i = 0; i < N; i++) {
      System.out.printf("%3d", arr[i] [j]);
   }
  System.out.println();
}
```

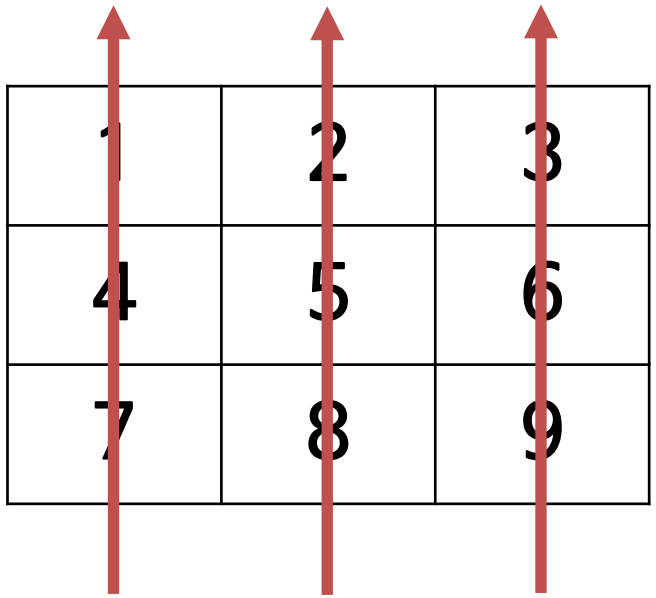
본 교안의 무단 배포 금지. copyright 2024 reserved. 20240919

배열 순회

탐색

- n x m 배열의 n * m 개의 모든 원소를 빠짐없이 조사하는 방법

- 열 역 우선 순회



열 역 우선 순회

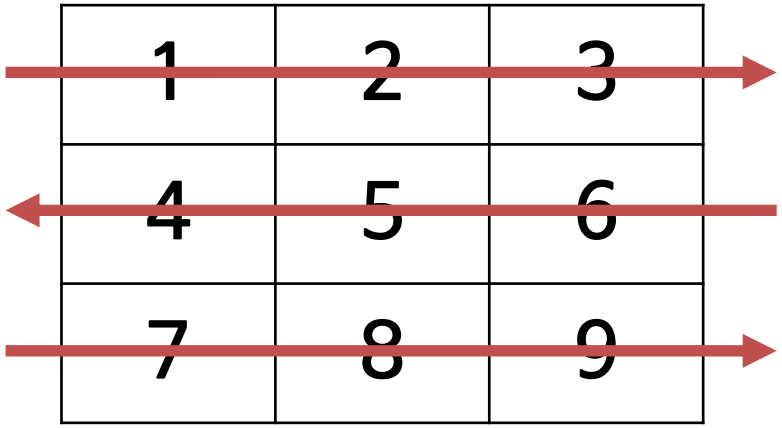
```
int i;          // 행의 좌표
int j;          // 열의 좌표

int[] [] arr = {{1,2,3},{4,5,6},{7,8,10}};

int N = arr.length;         // 행의 길이
int M = arr[0].length;      // 열의 길이

for(int j = 0; j < M; j++) {
   for(int i = M-1; i >= 0; i--) {
      System.out.printf("%3d", arr[i] [j]);
   }
   System.out.println();
}
```

본 교안의 무단 배포 금지
copyright 2024
20240919

배열 순회

탐색

▪ 지그재그 순회

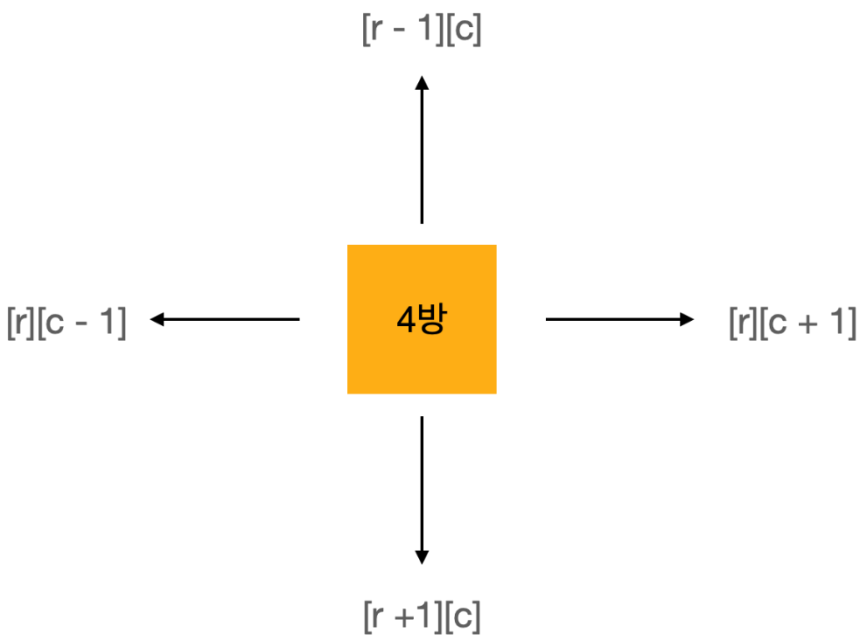


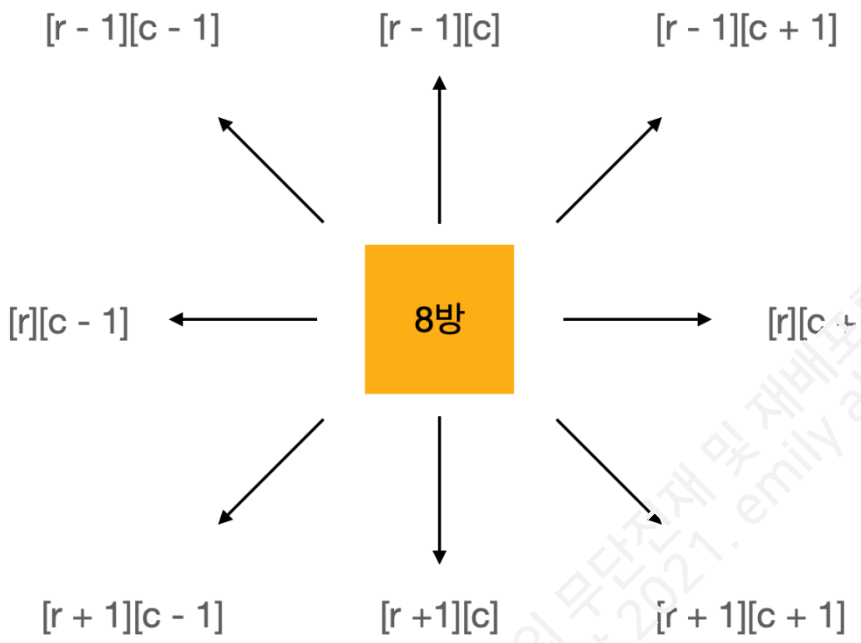
지그재그 순회

```
int i;          // 행의 좌표
int j;          // 열의 좌표

int[] [] arr = {{1,2,3},{4,5,6},{7,8,10}};

int N = arr.length;         // 행의 길이
int M = arr[0].length;      // 열의 길이

for(int i = 0; i < N; i++) {
  //정방향
  if(i % 2 == 0) {
    for(int j = 0; j < M; j++) {
      System.out.printf("%3d", arr[i] [j]);
    }
  } else {     // 역방향
    for(int j = M-1; j>=0;j--) {
      System.out.printf("%3d", arr[i] [j]);
    }
  }
  System.out.println();
}
```

델타 탐색

- **델타를 이용한 2차원 배열 탐색**
  - 2차원 배열의 한 좌표에서 4방향(혹은 8방향)의 인접 배열 요소를 탐색하는 방법

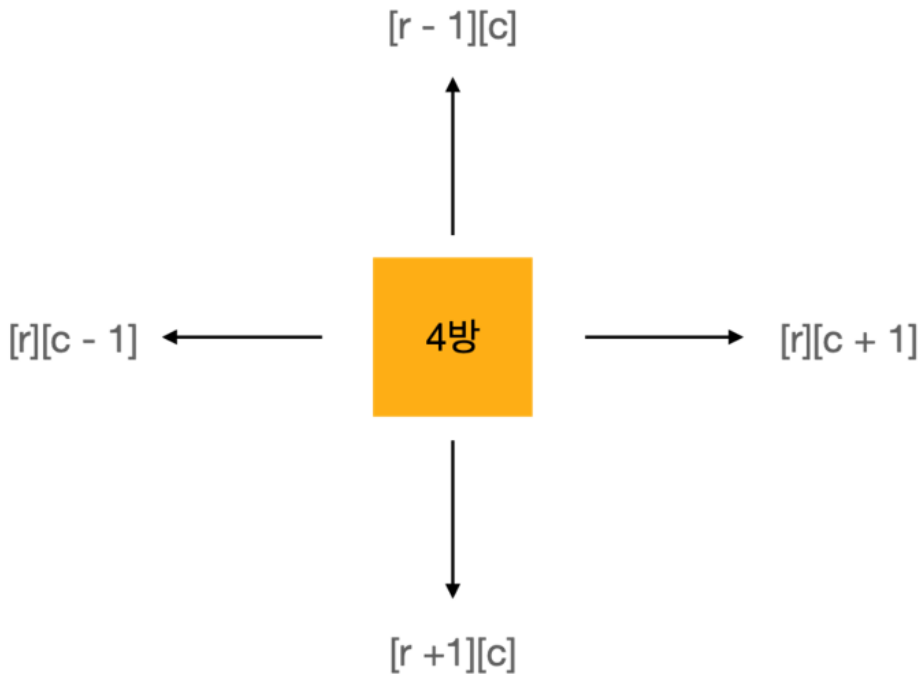




본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

델타 탐색

- **델타를 이용한 2차원 배열 탐색**
  - 4방 탐색

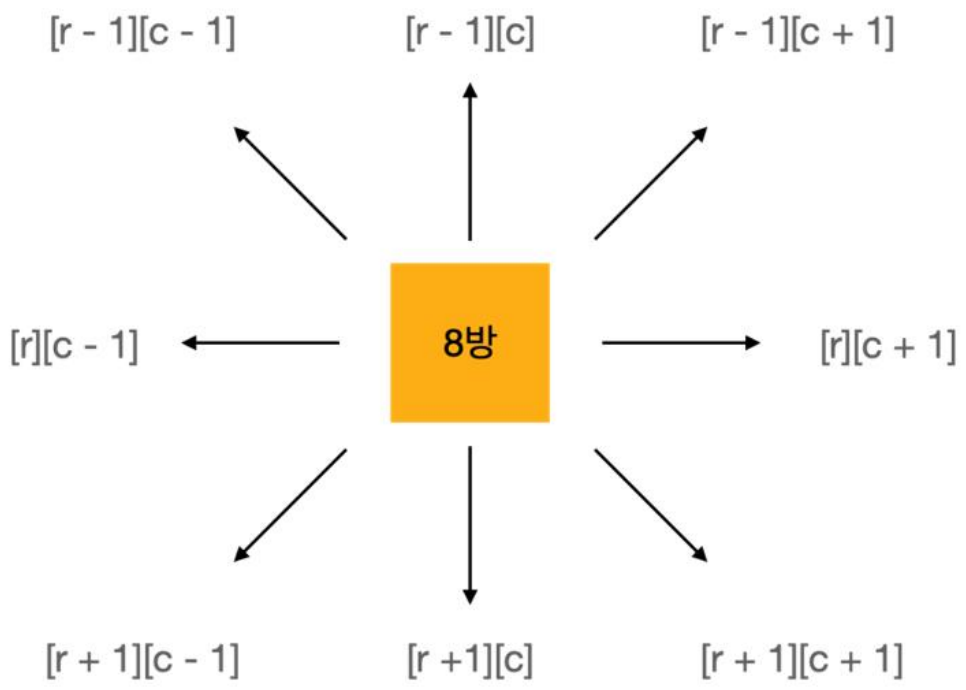


Search_Delta4.java

```
public class Search_Delta4 {
    public static void main(String[] args) {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,10}};
        // N*N
        int N = arr.length;
        // 현재 나의 좌표
        int r = 0, c = 0;
        // 상하좌우
        int[] dr = {-1, 1, 0, 0};
        int[] dc = {0, 0, -1, 1};
        for (int d=0;d<4;d++) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            // 위치에 따라 문제 발생 할 수 있다
            // 경계확인
            // 1. 범위 안
            if (nr >= 0 && nr < N && nc >=0 && nc < N) {
                System.out.println(arr[nr][nc]);
            }
            // 2. 범위 안이 아니면 안하겠다(버리기)
            if(nr < 0 || nr >= N || nc<0 || nc >=N ) continue;
            System.out.println(arr[nr][nc]);
        }
    }
}
```

델타 탐색

- **델타를 이용한 2차원 배열 탐색**
  - 8방 탐색



Search_Delta8.java

```
public class Search_Delta4 {
    public static void main(String[] args) {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,10}};
        // N*N
        int N = arr.length;
        // 현재 나의 좌표
        int r = 0, c = 0;
        // 상, 우상, 우, 우하, 하, 좌하, 좌, 좌상
        int[] dr = { -1, -1, 0, 1, 1, 1, 0, -1};
        int[] dc = {0, 1, 1, 1, 0, -1, -1, -1};
        for (int d=0;d<8;d++) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            // 위치에 따라 문제 발생 할 수 있다
            // 경계확인
            // 1. 범위 안
            if (nr >= 0 && nr < N && nc >=0 && nc < N) {
                System.out.println(arr[nr][nc]);
            }
            // 2. 범위 안이 아니면 안하겠다(버리기)
            if(nr < 0 || nr >= N || nc<0 || nc >=N ) continue;
            System.out.println(arr[nr][nc]);
        }
    }
}
```

multicampus

정 렬(II)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

카운팅 정렬(Counting Sorting)

- 항목들의 순서를 결정하기 위해 집합에 각 항목이 몇 개씩 있는지 세어서 적절한 위치에 선형 시간에 정렬하는 방법

- 각 항목의 개수를 기록하기 위해 정수로 인덱스 되는 카운트 리스트를 사용하기 때문에 정수나 정수로 표현할 수 있는 자료에만 적용할 수 있는 정렬 알고리즘
- 카운트들을 위한 충분한 공간을 할당하려면 집합 내의 가장 큰 정수를 알아야 함

- 시간복잡도 – O(n+k)
  n은 배열의 길이, k는 정수의 최대값

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

카운팅 정렬(Counting Sorting)

- {0, 4, 1, 3, 1, 2, 4, 1}

0	4	1	3	1	2	4	1

  - 최대 정수를 포함하는 배열을 생성
  - 각 정수의 개수를 세서 넣어 줌

1	3	1	1	2

  - 그 누적 합을 구함

[0]	[1]	[2]	[3]	[4]
1	4	5	6	8

  - 정렬 된 배열을 생성 한 후 뒤에서부터 읽어서 작성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

카운팅 정렬(Counting Sorting)

- **카운팅 정렬을 항상 사용하는 것이 좋을까**
  - 동떨어진 값은 카운팅 정렬하기에 좋지 않음

    예) {1,2,10억,3}
  - 밀집된 수에 사용하는 것이 좋음
- **카운팅 정렬은 안정정렬 이다**
  - 똑같은 값이라고 했을 때, 원래의 순서를 보장해주는 것
  - 같은 값을 가지는 복수의 원소들이 정렬 후에도 정렬 전과 같은 순서를 가지게 되는 것

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

카운팅 정렬(Counting Sorting)

- **복수의 원소 정렬 – 두 번째 원소를 기준으로 정렬**

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
(0, 3)	(4, 0)	(1, 2)	(3, 3)	(1, 4)	(2, 1)	(4, 1)	(1, 3)

- 각 정수의 개수를 세서 넣어 줌

1	2	1	3	1

- 그 누적 합을 구함

[0]	[1]	[2]	[3]	[4]
1	3	4	7	8

- 정렬 된 배열을 생성 한 후 뒤에서부터 읽어서 작성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
(4, 0)	(2, 1)	(4, 1)	(1, 2)	(0, 3)	(3, 3)	(1, 3)	(1, 4)

본 교재 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

카운팅 정렬(Counting Sorting)

- **복수의 원소 정렬 – 첫 번째 원소를 기준으로 정렬**

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
(4, 0)	(2, 1)	(4, 1)	(1, 2)	(0, 3)	(3, 3)	(1, 3)	(1, 4)

- 각 정수의 개수를 세서 넣어 줌

1	3	1	1	2

- 그 누적 합을 구함

[0]	[1]	[2]	[3]	[4]
1	4	5	6	8

- 정렬 된 배열을 생성 한 후 뒤에서부터 읽어서 작성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
(0, 3)	(1, 2)	(1, 3)	(1, 4)	(2, 1)	(3, 3)	(4, 0)	(4, 1)

본 자료의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

카운팅 정렬(Counting Sorting)

- **복수의 원소 정렬 - 완료**

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
(0, 3)	(1, 2)	(1, 3)	(1, 4)	(2, 1)	(3, 3)	(4, 0)	(4, 1)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

문 자 열

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

문자의 표현

- **컴퓨터의 문자표현**
  - 컴퓨터가 이해하는 언어는 0과 1 (bit). 2진수로 데이터를 처리.
    메모리는 컴퓨터가 이해하는 언어인 숫자로만 저장할 수 있기 때문에 문자는 직접 저장할 수가 없다.
    그래서 각 문자에 대응되는 숫자를 정한 뒤 메모리에 저장하는 방법이 사용된다.
  - 영어의 경우 대,소문자 합쳐서 52개의 글자를 6 (64가지 경우의 수) 비트면 모두 표현할 수 있다.
    이를 코드체계라 한다.
    ex) 000000 -〉 'a' , 000001 -〉 'b'
  - 네트워크가 발전되기 전까지 미국의 지역은 코드체계를 각각 정해 놓고 사용했지만, 네트워크가 발전하면서 서로 간에 정보를 주고 받을 때 정보의 해석이 달라진다는 문제가 생겼다.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

문자의 표현

문자열

컴퓨터의 문자표현

- 그 혼동을 피하기 위해 1967년 ASCII(American Standard Code for Information Interchange, 미국정보교환표준부호)라는 문자 인코딩 표준안을 제정하게 되었다.
- 아스키 코드 중 제어문자와 확장 아스키 코드를 제외한 부호

ASCII	부호	ASCII	부호	ASCII	부호	ASCII	부호
032		056	8	080	P	104	h
033	!	057	9	081	Q	105	i
034	"	058	:	082	R	106	j
035	#	059	;	083	S	107	k
036	$	060	〈	084	T	108	l
037	%	061	=	085	U	109	m
038	&	062	〉	086	V	110	n
039	'	063	?	087	W	111	o
040	(	064	@	088	X	112	p
041	)	065	A	089	Y	113	q
042	*	066	B	090	Z	114	r
043	+	067	C	091	[	115	s
044	,	068	D	092	₩	116	t
045	-	069	E	093	]	117	u
046	.	070	F	094	^	118	v
047	/	071	G	095	_	119	w
048	0	072	H	096	`	120	x
049	1	073	I	097	a	121	y
050	2	074	J	098	b	122	z
051	3	075	K	099	c	123	{
052	4	076	L	100	d	124	|
053	5	077	M	101	e	125	}
054	6	078	N	102	f	126	~
055	7	079	O	103	g		

본 교안의 모든 저작권 및 재배포를 금합니다.
copyright 2021. emily Inc. All rights reserved.

문자의 표현

- **컴퓨터의 문자표현**
  - ASCII는 7bit 인코딩으로 128문자를 표현하며 33개의 출력 불가능한 제어 문자들과 공백을 비롯한 95개의 출력 가능한 문자들로 이루어져 있음
  - 확장 ASCII는 8bit를 모두 사용함으로써 출력 불가능한 문자들까지 모두 표현할 수 있음

  - 인터넷이 전 세계로 발전하면서 각 국이 여러 개의 확장 아스키코드를 사용하다 보니 호환성 문제가 해결되지 않음
    서로 간의 정보 해석이 정확하게 이루어지지 않게 되면서 문제가 발생하게 됨
    그래서 다국어 처리를 위해 표준을 마련하게 되었고, 이를 유니코드(Unicode)라 함
    (오늘날에는 7비트 아스키 코드와 유니코드를 기준으로 삼는게 좋으며, 인코딩 방식은 UTF-8가 일반적임)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. eduly all rights reserved.
20240919

문자의 표현

- **유니코드(Unicode)**
  - 유니코드(Unicode)는 전 세계의 모든 문자를 컴퓨터에서 일관되게 표현하고 다룰 수 있도록 설계된 인코딩 표준
    1988년 초안이 탄생했고 현재는 유니코드 협회(Unicode Consortium)가 제정하며,
    유니코드의 목적은 현존하는 모든 문자 인코딩 방법들을 유니코드로 교체하려는 것
  - 유니코드는 표현상의 한계를 극복하기 위해 용량을 크게 확장해 2바이트(=16비트),
    즉 2의 16승인 65536개의 문자를 표현할 수 있음
    또한 고어나 희귀한 문자들은 2바이트를 초과할 수 있기에 유니코드 3.0부터는 보충언어판(BMP+SMP)을 정의해 약
    백만 자가 넘는 문자를 추가로 정의 함

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

문자의 표현

문자열

유니코드(Unicode)

- 유니코드 인코딩은 UTF-8, UTF-16, UTF-32 등이 있으며, 가장 일반적으로 UTF-8이 사용됨.
  영문, 숫자, 기호는 1바이트로, 한글, 한자는 3바이트로 표현.
  한글 유니코드는 한글 '가'가 유니코드로 AC00으로 시작된다.

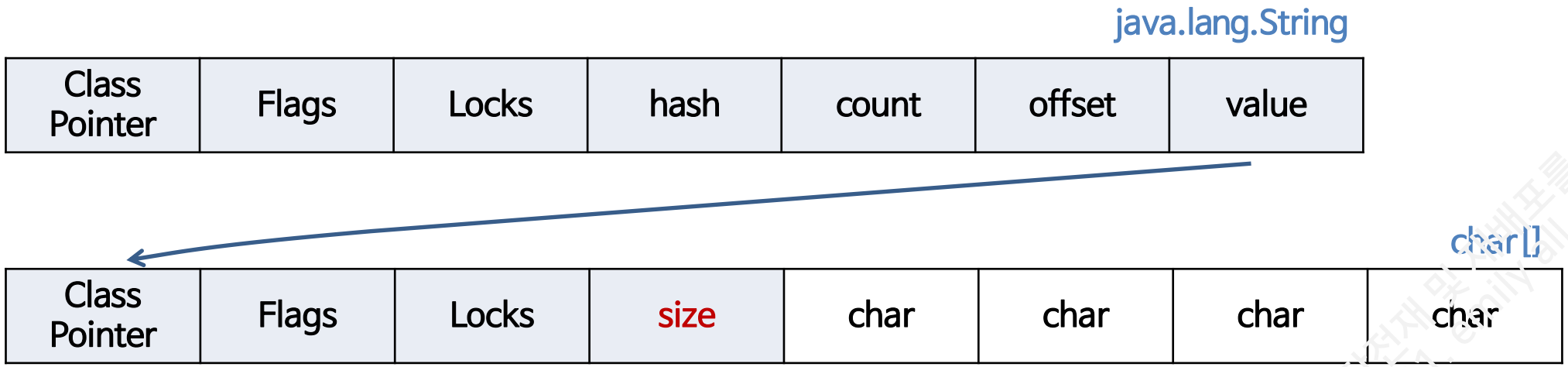
AC00 **Hangul Syllables** **ACFF**

	AC0	AC1	AC2	AC3	AC4	AC5	AC6	AC7	AC8	AC9	ACA	ACB	ACC	ACD	ACE	ACF
0	가	감	갠	갰	걀	걕	걤	거	검	겐	겠	결	겪	곁	고	곰
	AC00	AC10	AC20	AC30	AC40	AC50	AC60	AC70	AC80	AC90	ACA0	ACB0	ACC0	ACD0	ACE0	ACF0
1	각	갑	갢	갱	걁	걅	걥	걱	겁	겢	겡	겭	겹	곀	곡	곱
	AC01	AC11	AC21	AC31	AC41	AC51	AC61	AC71	AC81	AC91	ACA1	ACB1	ACC1	ACD1	ACE1	ACF1
2	갂	값	갣	갲	걂	걆	걦	걲	겂	겣	겢	겮	겺	곂	곢	곲
	AC02	AC12	AC22	AC32	AC42	AC52	AC62	AC72	AC82	AC92	ACA2	ACB2	ACC2	ACD2	ACE2	ACF2

본 자료는 무단 배포 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.

문자의 표현

- **java에서 String 클래스에 대한 메모리 배치 예시**
  - java.lang.String 클래스에는 기본적인 메타 데이터 외에도 네 가지 필드들이 포함되어 있는데, hash값(hash), 문자열의 길이(count), 문자열 데이터의 시작점(offset), 그리고 실제 문자열 배열에 대한 참조(value)



본 교안의 무단전재 및 배포를 금합니다.
copyright 2021. Daily all rights reserved.
20240919

문자의 표현

- Java에서의 문자열 처리
  - 문자열 데이터를 저장, 처리해주는 클래스를 제공
  - String 클래스를 사용

```
String str = “abc”;
String str = new String(“abc”)’
```

  - 문자열 처리에 필요한 연산을 연산자, 메소드 형태로 제공
    - +, length(), replace(), split(), substring(), …
    - 보다 풍부한 연산을 제공

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

문자의 표현

- **문자열 비교**
  - equals() 메소드 제공 – == 연산은 메모리 참조가 같은지를 묻는 것

- **문자열 숫자를 정수형으로 반환**
  - 숫자 클래스의 parse 메소드를 제공
  - 예 – Integer.parseInt(String)
  - 역함수로는 toString() 메소드를 제공

```
int atoi(char[] charArr) {
        int value = 0;
        int digit;
        for(int i = 0;i < charArr.length; i++) {
                if(charArr[i] >= '0' && charArr[i] <= '9') {
                        digit = charArr[i] - '0';
                else{
                        break;
                }
                value = (value * 10) + digit;
        }
        return value;
}
```

copyright 2024

multicampus

패 턴 매 칭

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

패턴매칭

- **패턴 매칭(문자열 매칭, Pattern Matching)**
  - 데이터에서 문자열을 검색할 때 패턴 매칭 알고리즘을 사용하여 검색

- **종류**
  - 고지식한 패턴 검색 알고리즘(Brute-Force Algorithm)

    본문 문자열을 처음부터 끝까지 차례대로 순회하면서 패턴 내의 문자들을 일일이 비교
  - 카프-라빈 알고리즘(Rabin-Karp Algorithm)

    고지식한 패턴 검색 알고리즘과 같이 하나씩 문자를 이동하며 일치하는지 확인하지만, 해시 값을 사용
  - KMP 알고리즘(Knuth-Morris-Pratt(KMP) Algorithm)
  - 보이어-무어 알고리즘(Boyer-Moore Algorithm)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

고지식한 패턴 검색 알고리즘(Brute-Force Algorithm)

- 본문 문자열을 처음부터 끝까지 차례대로 순회하면서 패턴 내의 문자들을 일일이 비교하는 방식으로 동작

0
Given text string | Brute-Force Algorithm is easy

0
Pattern string | Force

1
Given text string | Brute-Force Algorithm is easy

0
Pattern string | Force

6
Given text string | Brute-Force Algorithm is easy

0
Pattern string | Force

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. multicampus all rights reserved.
20240919

고지식한 패턴 검색 알고리즘(Brute-Force Algorithm)

- 본문 문자열을 처음부터 끝까지 차례대로 순회하면서 패턴 내의 문자들을 일일이 비교하는 방식으로 동작

6 7

Given text string | Brute-Force Algorithm is easy

0 1

Pattern string | Force

6 7 8 9 10

Given text string | Brute-Force Algorithm is easy

0 1 2 3 4

Pattern string | Force

7

Given text string | Brute-Force Algorithm is easy

0

Pattern string | Force

본 교안의 무단 전재 및 재배포를 금합니다.
copyright 2021 family all rights reserved.
20240919

multicampus

스 택

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

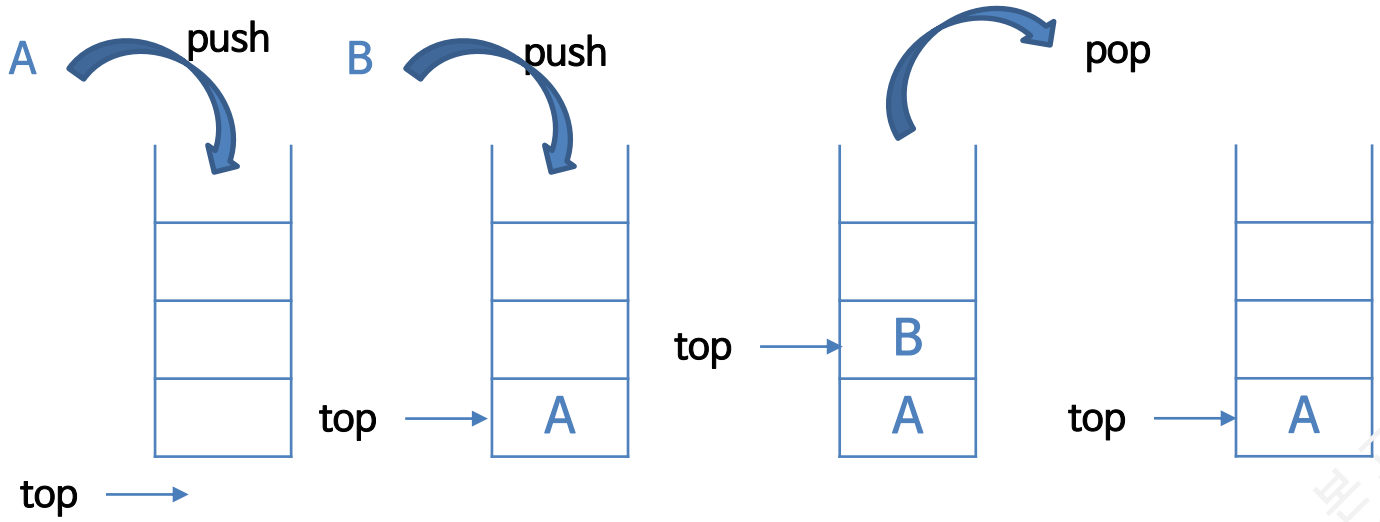
스택(Stack)

- 데이터를 쌓아서 관리하는 방식
- 삽입과 삭제 연산이 후입선출(LIFO, Last-In-First-Out)로 이루어진 자료구조
  - 마지막에 삽입한 자료를 가장 먼저 꺼내는 것
- 스택에 저장된 자료는 선형구조를 가짐
  - 선형구조 : 자료 간의 관계가 1대1
  - 비선형구조 : 자료 간의 관계가 1대 N
- 스택을 프로그램에서 구현하기 위해 자료구조가 필요
  - 자료구조 – 자료를 선형으로 저장할 저장소
    - 저장소 자체를 스택이라 부르기도 함

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

스택(Stack)

- **스택의 연산 - 삽입, 삭제, 스택의 공백여부**
  - top - 삽입과 삭제가 일어나는 위치
  - push - top 위치에 새로운 데이터를 삽입
  - pop - top 위치에 현재 있는 데이터를 삭제하고 확인
  - peek - top 위치에 현재 있는 데이터를 확인



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

스택(Stack)

- **장점**
  - **간결한 구조** : 간단하고 직관적인 구조를 가지고 있음. push, pop 연산을 사용하여 간단함.
  - **빠른 삽입과 삭제** : 스택의 삽입(push)과 삭제(pop) 연산은 상수 시간(O(1))에 수행. 스택이 데이터를 위쪽에만 삽입하고 삭제하기 때문에 발생하는 장점.
  - **재귀호출** : 재귀 호출과 함께 사용 됨. 함수가 호출될 때마다 호출 스택에 저장되고, 재귀 문제 해결을 가능하게 함.
  - **역순 데이터 열람** : 데이터를 역순으로 열람하는 데 유용. 문자열을 뒤집을 때 스택 사용이 가능.

- **단점**
  - **크기 제한** : 데이터 최대 개수를 미리 정해야 함. 한계에 도달 시 요소 저장이 불가함.
  - **검색의 어려움** : 특정 위치의 데이터에 직접 접근하는 것이 어려움. 매번 pop하면서 요소를 찾아야 함.
  - **비효율적인 공간 사용** : 연속된 메모리 공간을 사용하기 때문에, 일부 공간이 낭비될 수 있음.
  - **제한된 용도** : 주로 선입선출(LIFO) 데이터 구조로 사용되어 그 외 다른 용도로 사용하기에는 적합하지 않을 수 있음.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240915

multicampus

재 귀 함 수

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

재귀

- **재귀(再歸, Recursion) - 자신을 정의할 때 자기 자신을 재참조 하는 방법**
  - 함수가 진행되는 동안 하위 작업을 수행하기 위해 자신을 여러 번 호출하는 데 사용.
    재귀 함수는 문제를 더 작은 하위 문제로 분할하여 해결하는데 적합한 상황에서 사용 됨.
    반복 구조에 비해 이해하기 쉽고 빠르게 구현이 가능
  - 함수 호출은 스택 메모리를 사용 (이름만 같은 다른 메서드)
    간단한 문제에 대해서는 반복문에 비해 메모리 및 속도에서 성능저하 발생

- **재귀함수의 구조**
  - Base Case : 재귀 함수의 호출을 멈추는 조건을 설정. 함수가 무한히 호출되는 것을 방지하며, 기본 경우에 도달하면 재귀가 멈춤.
  - Recursive Case : 기본 경우가 아닌 경우, 함수는 자기 자신을 다시 호출하며 더 작은 입력을 이용하여 작업을 수행 함. Base Case를 호출할 수 있도록 유도해야 함.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. eny all rights reserved.
20240919

재귀

재귀함수

- **장점**
  - 코드의 간결성과 가독성을 높일 수 있음
- **단점**
  - 잘못 구현하거나 기본 경우를 처리하지 않으면 무한 재귀에 빠질 수 있음

- 분할정복 알고리즘에서 유용하게 사용 됨
  (예:피보나치 수열, 팩토리얼 계산, 하노이의 탑 등)

Recursive.java

```
public class Recursive {
    public static void main(String[] args) {
        func(3);
    }

    public static void func2(int n) {
        // base case
        if(n == 0) {
            System.out.println("n은 0이다");
            return;
        }
        // recursive case
        func(n-1);
    }
}
```

재귀호출

- **함수에서 실행해야 하는 작업의 특성에 따라 일반적인 호출방법보다 재귀호출을 사용하여 함수를 만들면 프로그램의 크기를 줄이고 간단히 작성**
  - factorial : 1부터 n까지의 모든 자연수를 곱하여 구하는 연산

```
n! = n x (n-1)!
(n-1)! = (n-1) x (n-2)!
(n-2)! = (n-2) x (n-3)!
...
ex) 4! = 1 x 2 x 3 x 4
4! = 4 x 3!
3! = 3 x 2!
2! = 2 x 1!
1! = 1
```

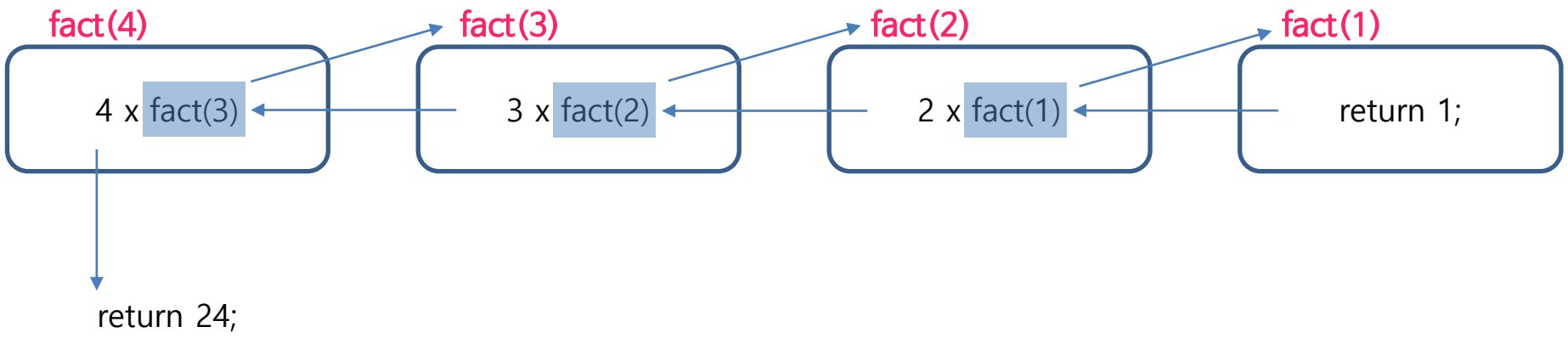
마지막에 구한 하위 값을 이용하여 상위값을 구하는 작업을 반복하게 됨

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

재귀호출

- n = 4 인 경우

```
4! = 4 x 3!
3! = 3 x 2!
2! = 2 x 1!
1! = 1
```



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

동적 계획법과 메모이제이션

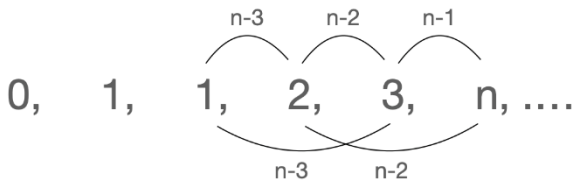
- **동적 계획법(Dynamic Programming)**
  - 큰 문제를 작은 하위 문제로 나누어 해결한 후, 하위 문제의 해를 이용하여 전체 문제의 해를 구하는 방식
  - 작은 하위 문제의 해를 계산한 후, 이를 저장해 두고 이용하여 중복 계산을 피함
  - 일반적으로 하향식(Top-down)과 상향식(Bottom-up) 두 가지 접근 방식이 있음
  - 주로 최적화 문제를 푸는데 사용되며, 다이나믹 프로그래밍의 다양한 유형이 존재함
- **메모이제이션(Memoization)**
  - 동적 계획법의 하향식(Top-down) 접근 방식 중 하나
  - 함수 호출의 결과를 저장하여, 같은 입력에 대한 함수 호출을 다시 하지 않도록 하여 중복 계산을 줄임
  - 대부분 재귀적으로 문제를 해결할 때 적용

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

재귀호출

- **피보나치 수열 - 0과 1로 시작하고 바로 직전 두 수의 합을 다음 항으로 하는 수열**
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

```
n = (n-1) + (n-2)
(n-1) = (n-2) + (n-3)
(n-2) = (n-3) + (n-4)
```



  - 피보나치 수열의 n번째 값을 계산하는 함수

$$F_n := \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n > 1. \end{cases}$$

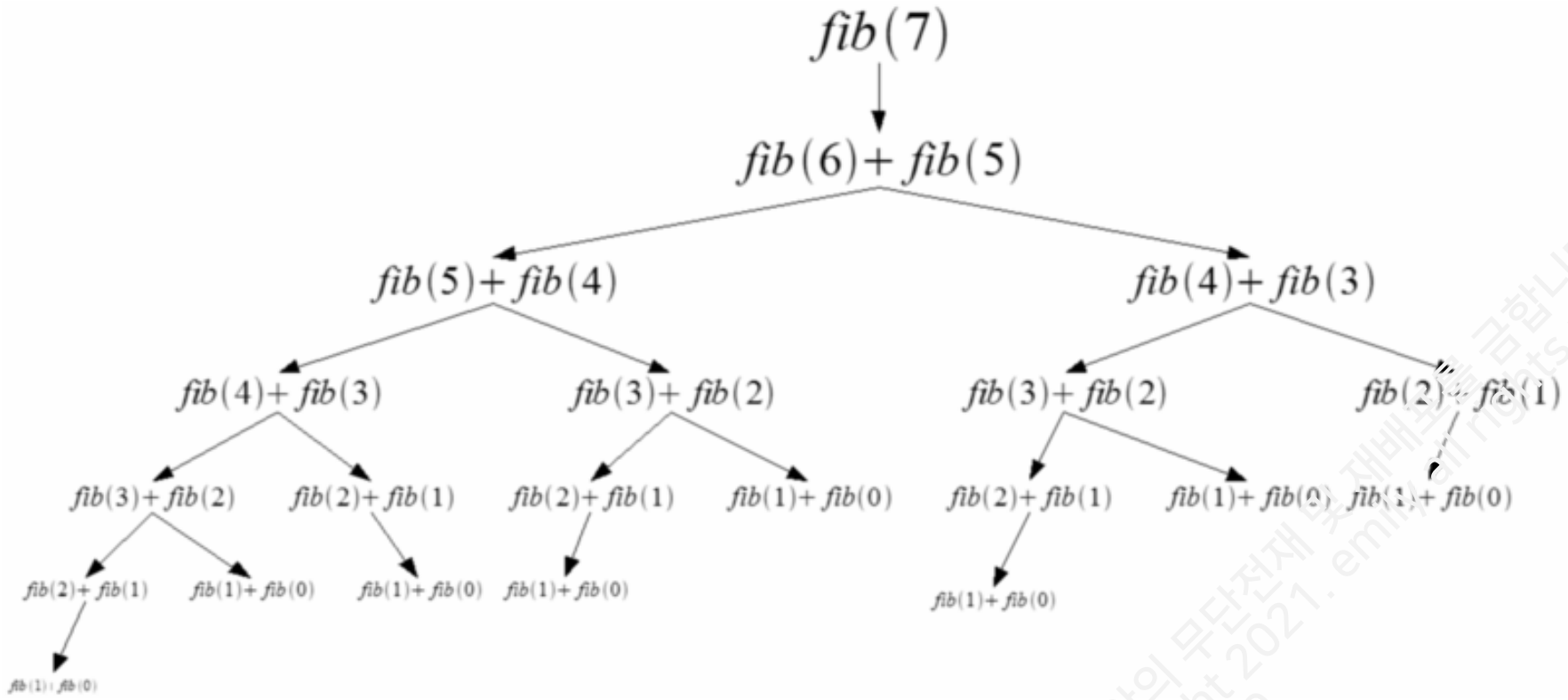
  - 피보나치 수열의 n번째 항을 반환하는 함수를 재귀함수로 구현 가능

```
public static int fibo(int n) {
        if (n <= 1) {
                return n;
        } else {
                return fibo(n-1) + fibo(n-2);
        }
}
```

재배포를 금합니다.
all rights reserved.

재귀호출

- **재귀호출의 문제점**
  - 중복호출이 존재

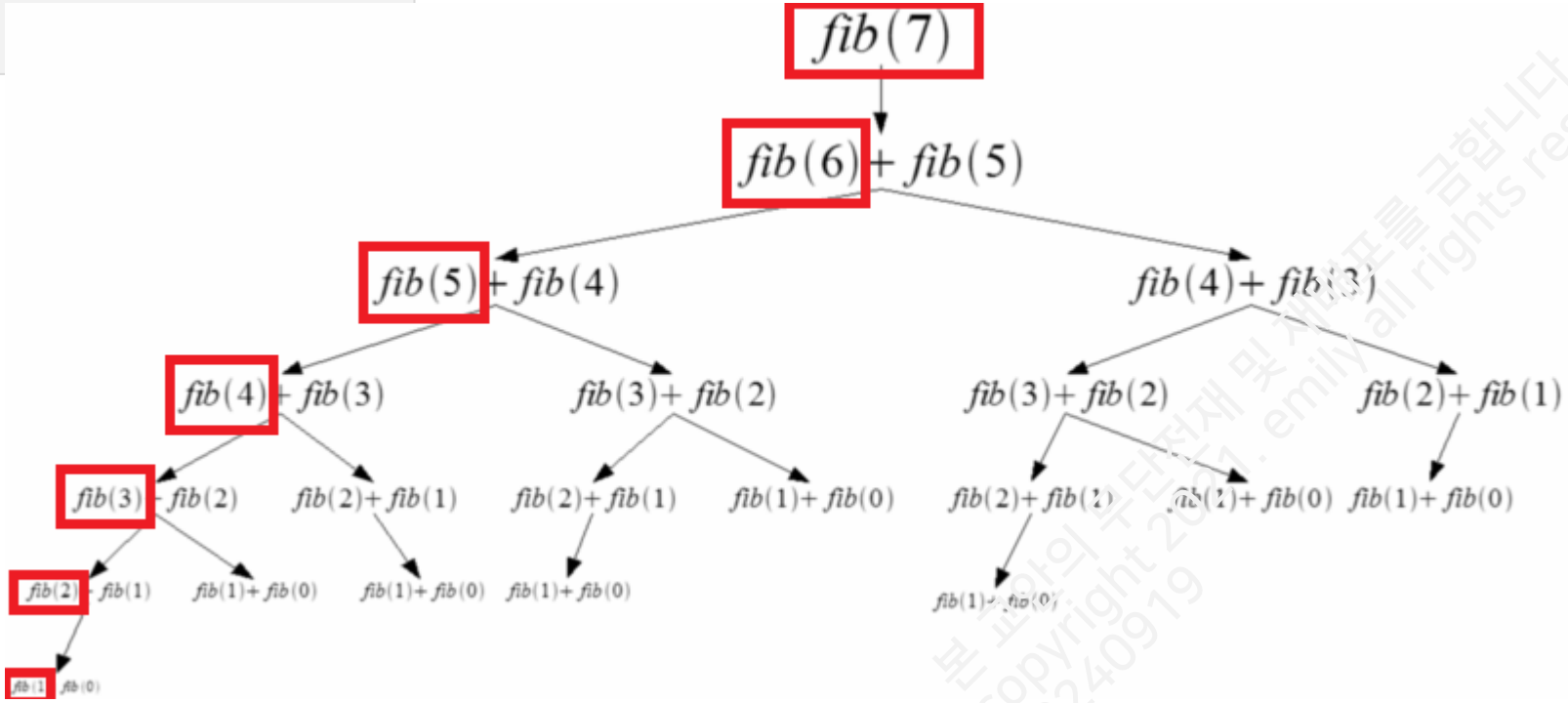


본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emilkim rights reserved.
20240919

재귀호출

- 메모이제이션(memorization) - 메모리에 저장해서 매번 다시 계산하지 않도록 하여 실행속도를 빠르게 함

```
public static int mfibo(int n) {
        if (n >= 2 && memo[n] == 0) {
        memo[n] = mfibo(n-1) + mfibo(n-2);
    }
    return memo[n];
}
```



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

큐

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

큐(Queue)

큐

- 먼저 입력한 데이터가 먼저 출력되는 특징을 가진 자료구조
- 삽입과 삭제 연산이 선입선출(FIFO, Fast-In-First-Out)로 이루어진 자료구조
  - 스택과 다르게 먼저 들어온 데이터가 먼저 나감
  - 한쪽 끝에서는 삽입 작업이 이루어지고 반대쪽 끝에서는 삭제 작업으로 이루어짐
  - 데이터를 순차적으로 처리해야 하는 경우에 유용함

- 스택과 마찬가지로 삽입, 삭제의 위치가 제한적인 자료구조

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

큐(Queue)

- **큐의 구조**
  - rear – 큐에서 가장 끝 데이터를 가리킴
  - front – 큐에서 가장 앞의 데이터를 가리킴
  - add – rear 부분에 새로운 데이터를 삽입
  - poll – front 부분에 있는 데이터를 삭제하고 확인
  - peek – 큐의 맨 앞에 있는 데이터를 확인

- **큐의 연산**
  - enQueue – 삽입 연산
  - deQueue – 삭제 연산

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. enjoy all rights reserved.
20240919

큐(Queue)

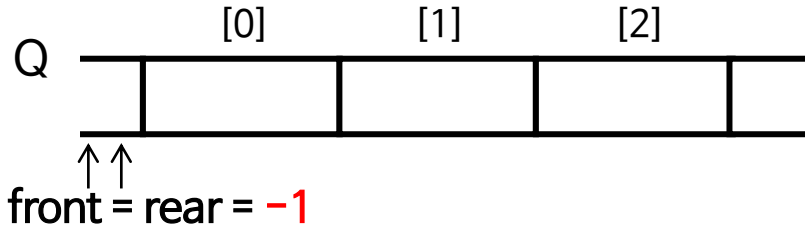
큐의 사용을 위한 주요 연산

연산	기능
enQueue(item)	큐의 뒤쪽(rear+1)에 원소를 삽입하는 연산
deQueue()	큐의 앞쪽(front)에서 원소를 삭제하고 반환하는 연산
createQueue()	공백 상태의 큐를 생성하는 연산
isEmpty()	큐가 공백상태인지를 확인하는 연산. front = rear 면 큐가 빈 것으로 판단
isFull()	큐가 포화상태인지를 확인하는 연산. rear가 마지막 인덱스를 가리키면 꽉 찬 것
Qpeek()	큐의 앞쪽에서 원소를 삭제 없이 반환하는 연산

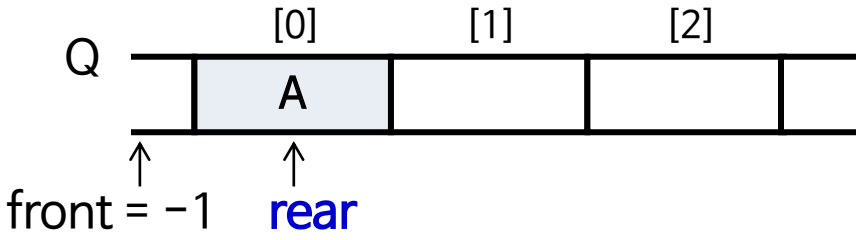
본 교안의 무단전재 및 배포를 금합니다.
copyright 2021. emm all rights reserved.
20240919

큐(Queue)의 연산

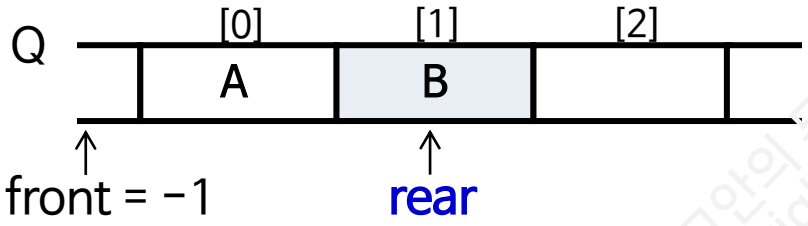
- 공백 큐 생성 - createQueue();



- 원소 A 삽입 - enQueue(A);



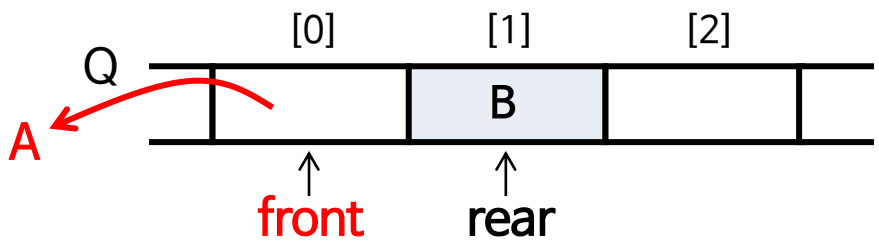
- 원소 B 삽입 - enQueue(B);



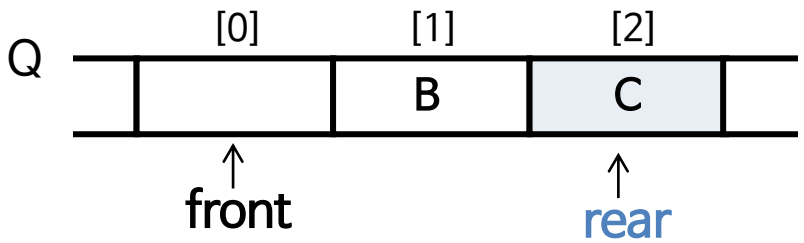
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

큐(Queue)의 연산

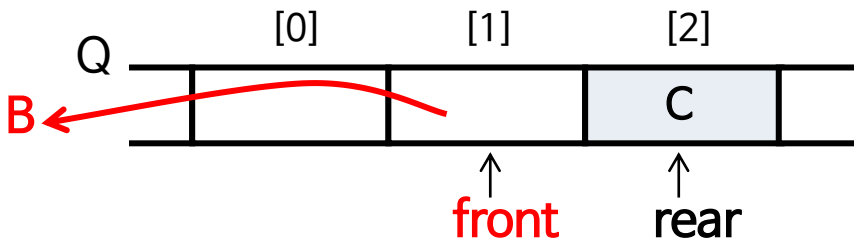
- 원소 반환/삭제 - deQueue();



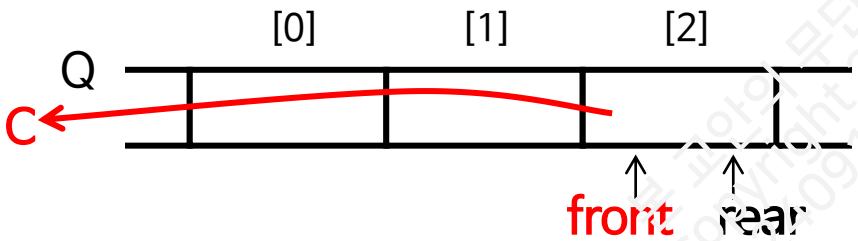
- 원소 C 삽입 - enQueue();



- 원소 반환 / 삭제 - deQueue();



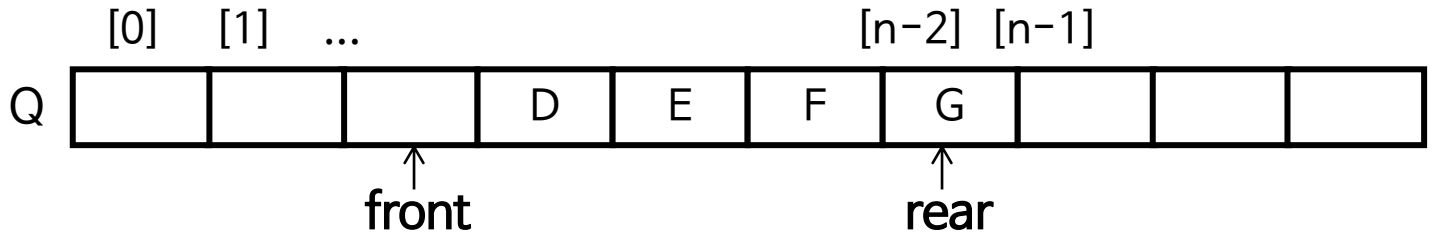
- 원소 반환 / 삭제 - deQueue();



본 교안은 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
2021.09.19

선형 큐(Linear Queue)

- **선형 큐 - 1차원 배열을 이용한 큐**
  - 큐의 크기 = 배열의 크기
  - front - 저장된 첫 번째 원소 바로 앞 인덱스
  - rear - 저장된 마지막 원소의 인덱스



- **상태 표현**
  - 초기 상태 -〉 front = rear = -1
  - 공백 상태 -〉 front = rear
  - 포화 상태 -〉 rear = n-1 (n : 배열의 크기, n-1 : 배열의 마지막 인덱스)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue)

- **큐 생성**
  - 크기 n인 1차원 배열 생성
  - front와 rear를 -1로 초기화

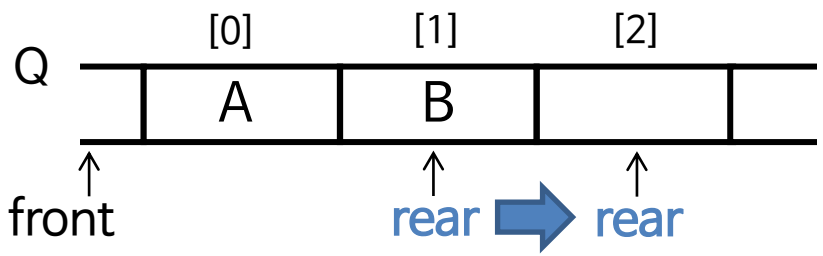
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

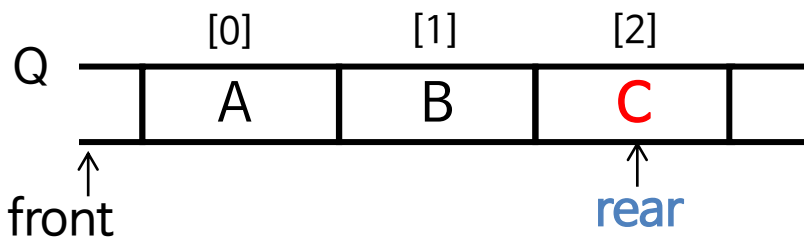
선형 큐(Linear Queue)

- 삽입 - enQueue(item)
  - 마지막 원소 뒤에 새로운 원소를 삽입해야 하므로

    ① rear값을 하나 증가시켜 삽입할 자리를 준비

    ② rear위치에 item을 저장






```
enQueue(item) {
        if(isFull()) print("Queue_Full");
        else {
                rear ← rear+1;          // ①
                Q[rear] ← item;         // ②
        }
}
```

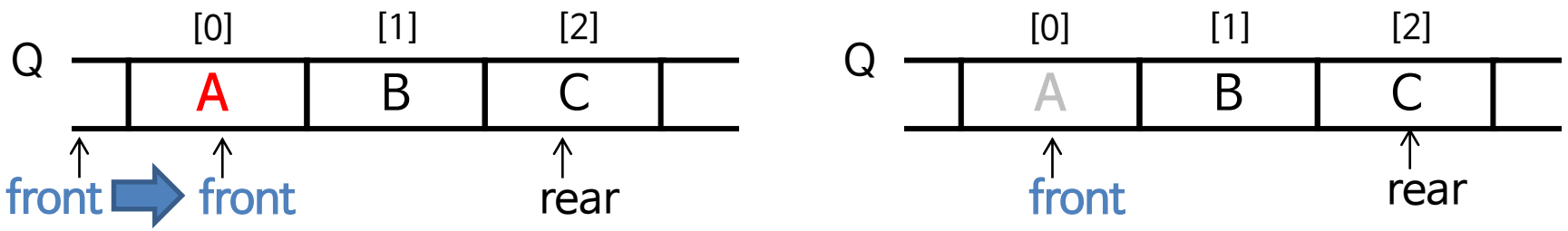
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue)

- 삭제 - deQueue()
  - 가장 앞에 있는 원소를 삭제해야 하므로

    ① front의 위치를 한자리 뒤로 이동하여 큐에 남아있는 첫 번째 원소의 위치로 이동

    ② 새로운 첫 번째 원소를 리턴 함으로서 삭제와 동일한 기능을 하게 됨




```
deQueue() {
        if(isEmpty(Q)) print("Queue_Empty");
        else {
                front ← front+1;        // ①
                return Q[front];        // ②
        }
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue)

- **공백상태 및 포화상태 검사 - isEmpty(), isFull()**
  - 공백상태 - front = rear
  - 포화상태 - rear = n-1 (n : 배열의 크기, n-1 : 배열의 마지막 인덱스)

```
isEmpty() {
        if(front == rear) return true;
        else return false;
}

isFull() {
        if(rear = n-1) return true;
        else return false;
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue)

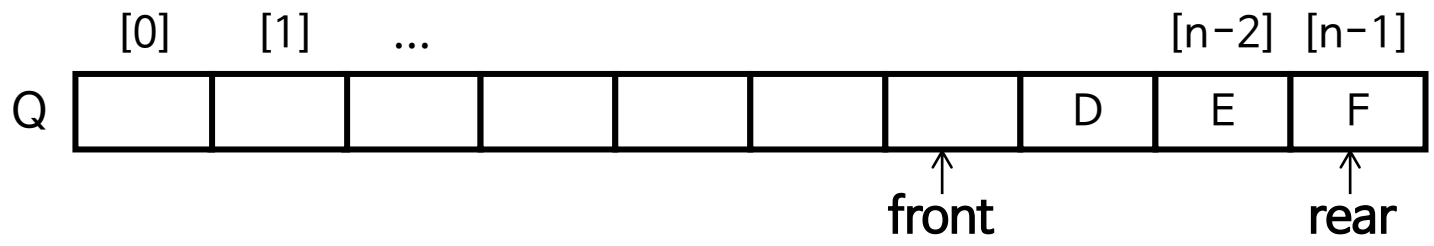
- 검색 - Qpeek()
  - 가장 앞에 있는 원소를 검색하여 반환하는 연산
  - 현재 front의 한자리 뒤(front+1)에 있는 원소, 즉 큐의 첫 번째에 있는 원소를 반환

```
Qpeek() {
        if(isEmpty()) print("Queue_Empty");
        else return Q[front+1]
}
```

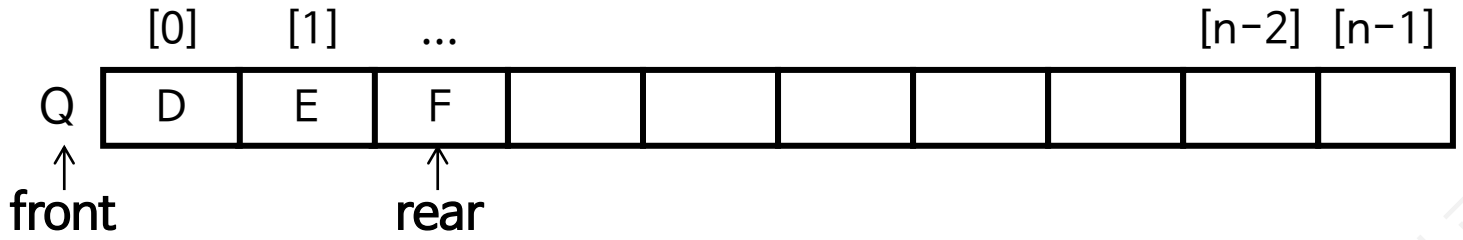
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue) 이용시의 문제점

- **선형 큐의 포화상태 문제**
  - 삽입과 삭제를 반복하면서 배열의 앞부분에 활용할 수 있는 공간이 있지만 포화상태(rear=n-1)로 인식하여 더 이상의 삽입을 수행하지 않게 됨



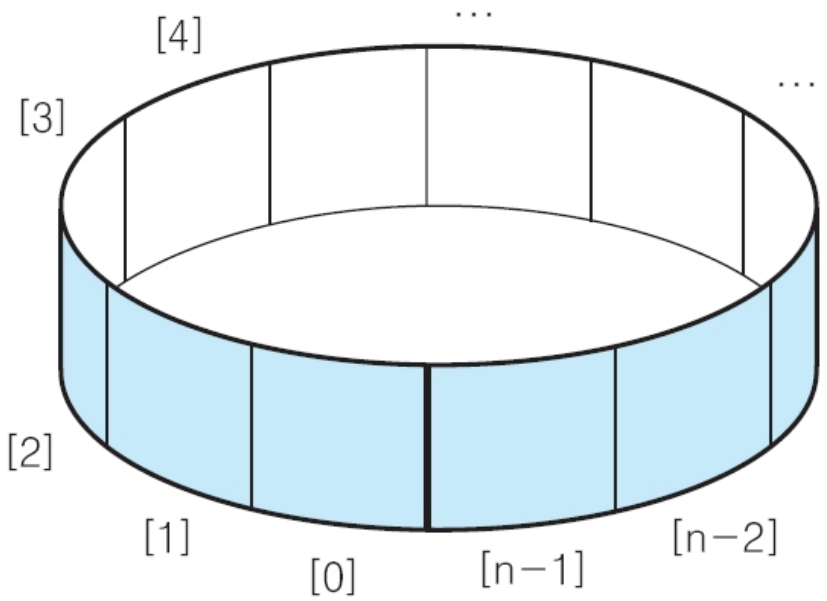
  - 해결방법1
    - 연산이 이루어질 때마다 원소들을 배열의 앞부분으로 모두 이동시킴
    - 순차적인 자료 이동 작업이 이루어지면서 시간 소요가 많이 일어나면서 큐의 효율성이 급격히 떨어짐



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

선형 큐(Linear Queue) 이용시의 문제점

- **선형 큐의 포화상태 문제**
  - 해결방법2
    - 1차원 배열을 사용하되 논리적으로 배열의 처음과 끝이 연결되어 있다고 가정하고 사용
    - 원형 큐의 논리적 구조



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

- **초기 공백 상태**
  - front = rear = 0
  - 큐 연산 시 front와 rear의 위치를 이동

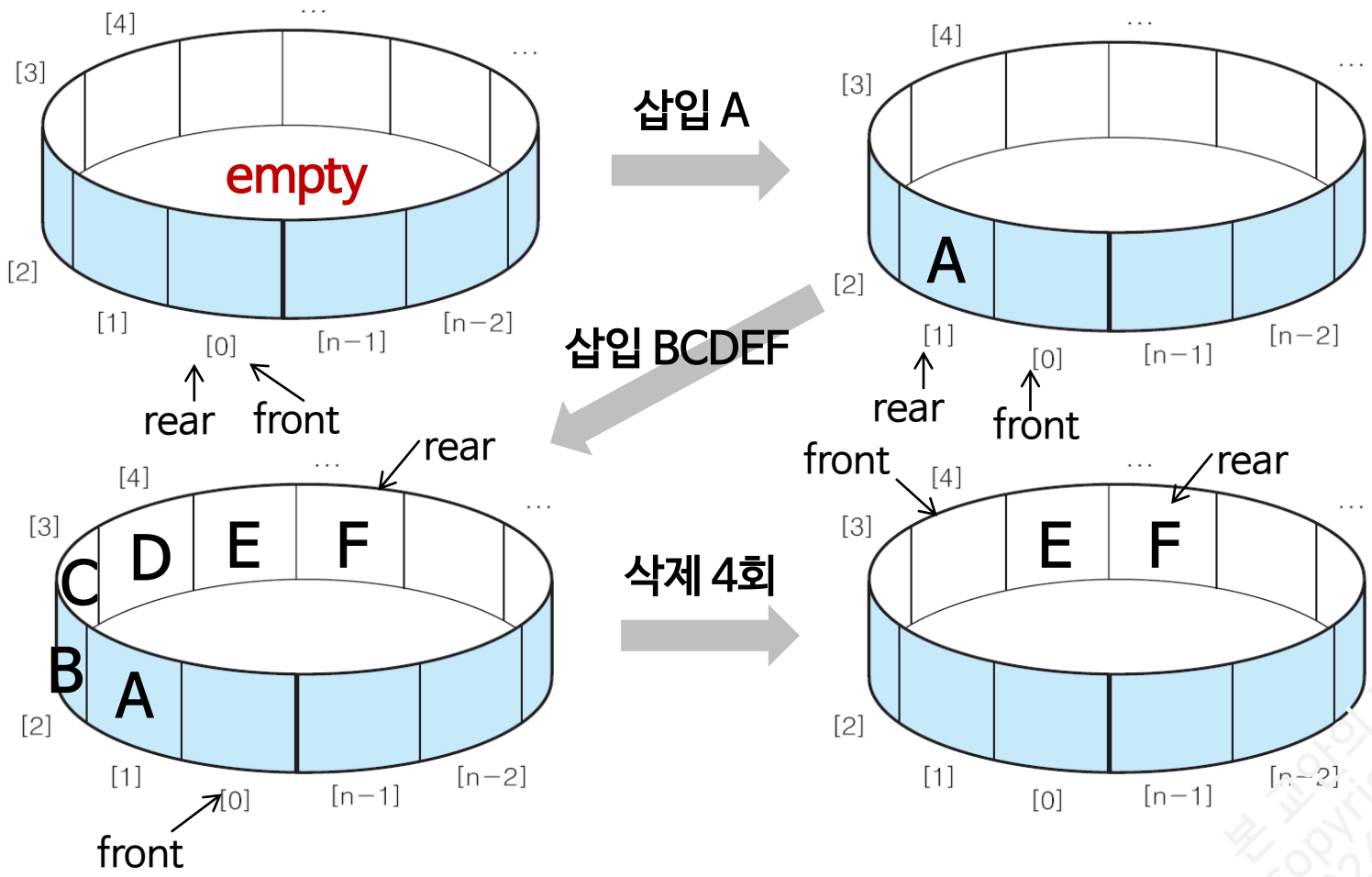
    배열의 마지막 인덱스인 n-1 다음 인덱스인 0번으로 이동하기 위해 나머지 연산자 mod를 사용

	삽입 위치	삭제 위치
선형 큐	rear = rear + 1	front = front + 1
원형 큐	rear = (rear + 1) mod n	front = (front + 1) mod n

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

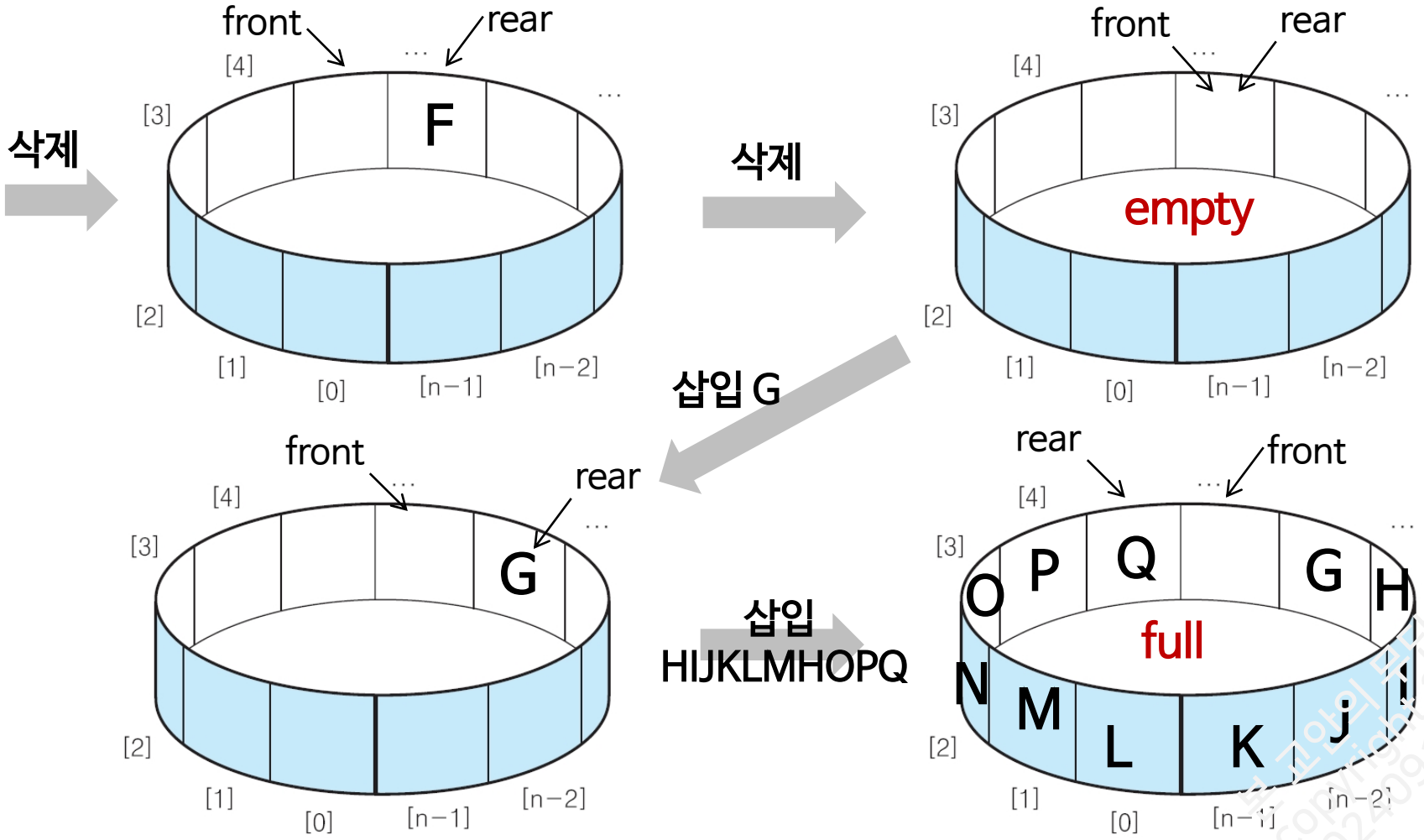
- 공백 상태와 포화 상태 구분을 쉽게 하기 위해서 front가 있는 자리는 사용하지 않고 항상 빈자리로 둠



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

- 공백 상태와 포화 상태 구분을 쉽게 하기 위해서 front가 있는 자리는 사용하지 않고 항상 빈자리로 둠



본 강의 내용은 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
2024.09.19

원형 큐(Circular Queue)

- **큐 생성**
  - 크기 n인 1차원 배열 생성
  - front와 rear를 0으로 초기화

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

- 삽입 - enQueue(item)
  - 마지막 원소 뒤에 새로운 원소를 삽입해야 하므로

    ① rear의 값을 조정하여 삽입할 자리를 준비 : rear ← (rear+1) mod n;

    ② 그 자리인 cQ[rear]에 원소 item을 삽입

```
enQueue(item) {
        if(isFull()) print("Queue_Full");
        else {
                rear ← (rear + 1) mod n ;        // ①
                cQ[rear] ← item;                 // ②
        }
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

- 삭제 - deQueue(), delete()
  - 가장 앞에 있는 원소를 삭제해야 하므로

    ① front의 값을 조정하여 삭제할 자리를 준비 : front ← (front+1) mod n;

    ② 새로운 front 원소를 리턴 함으로서 삭제와 동일한 기능을 하게 됨

```
deQueue() {
        if(isEmpty()) print("Queue_Empty");
        else {
                front ← (front + 1) mod n ;        // ①
                return cQ[front];                  // ②
        }
}
```

```
delete() {
        if(isEmpty()) print("Queue_Empty");
        else  front ← (front + 1) mod n ;    // ①
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

원형 큐(Circular Queue)

- **공백상태 및 포화상태 검사 - isEmpty(), isFull()**
  - 공백상태 - front = rear
  - 포화상태 - 삽입할 rear의 다음 위치 = 현재 front

    (rear + 1) mod n = front

```
isEmpty() {
        if(front == rear) return true;
        else return false;
}

isFull() {
        if((rear + 1) mod n == front) return true;
        else return false;
}
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

우선순위 큐(Priority Queue)

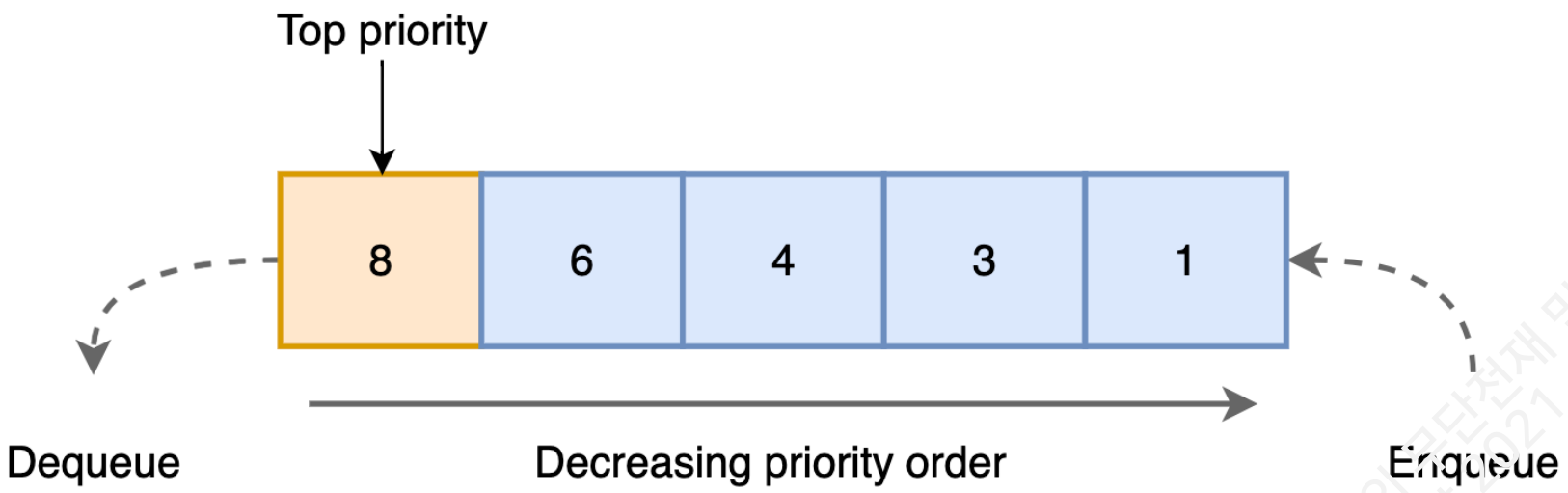
- 우선순위 항목들을 저장하는 큐
- 일반적인 큐는 먼저 넣은 데이터가 먼저 나오는 형태(FIFO) 구조로 되어 있으나,
  우선순위 큐는 들어간 순서에 상관없이 우선 순위가 높은 데이터가 먼저 나오는 것을 말함
- 운영체제의 작업 스케줄링, 정렬, 네트워크 관리 등의 다양한 기술에 적용되고 있음

- 우선순위 큐의 구현
  - 선형 배열을 이용한 큐
  - 연결 리스트를 이용한 큐

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

우선순위 큐(Priority Queue)

- **기본 연산**
  - enqueue() - 새로운 요소 삽입
  - dequeue() - 최대 우선순위 요소(top priority)를 삭제하고 그 값을 리턴
  - peek() - 최대 우선순위 요소 리턴



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

우선순위 큐(Priority Queue)

- **장점**
  - **우선순위 지원**: 가장 큰 장점은 우선순위를 지원한다는 것. 데이터를 저장할 때 각 데이터에 우선순위 값을 부여할 수 있으며, 이를 기반으로 우선순위가 높은 데이터가 먼저 처리 됨.
  - **빠른 삽입과 삭제**: 우선순위 큐의 삽입 및 삭제 연산은 효율적으로 수행됨. 최소 힙(Min Heap) 또는 최대 힙(Max Heap)과 같은 특수한 데이터 구조를 사용하면 삽입 및 삭제 연산이 O(log n) 시간에 이루어짐
  - **다양한 응용 분야**: 우선순위 큐는 다양한 응용 분야에서 사용됨.
    예: 작업 스케줄링, 그래프 알고리즘(최소 스패닝 트리, 다익스트라 알고리즘 등), 데이터 압축, 네트워크 라우팅, 시뮬레이션 등
  - **최소/최대 값 쉽게 접근**: 최소값 또는 최대값에 쉽게 접근이 가능. 최소 힙을 사용하면 최소값에, 최대 힙을 사용하면 최대값에 빠르게 접근이 가능.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

우선순위 큐(Priority Queue)

- **단점**
  - **동일 우선순위 처리 어려움**: 동일한 우선순위를 가진 데이터를 처리하는 경우, 일반적인 우선순위 큐는 입력 순서에 따라 처리하게 되어서 정확한 순서를 보장하지 않음.
  - **우선순위 함수 설계 어려움**: 적절한 우선순위 함수(비교 함수)를 설계하는 것이 중요함. 잘못된 우선순위 함수로 인해 예상치 못한 결과가 발생할 수 있음.
  - **데이터 중복 처리 어려움**: 일반적으로 우선순위 큐는 중복된 데이터를 처리하기 어려워서 추가적인 구현이 필요함.
  - **구현 복잡성**: 우선순위 큐는 효율적인 구현을 위해 힙(Heap)과 같은 특별한 자료구조를 사용함. 이러한 자료구조를 이해하고 구현하는 데는 일정한 학습 곡선이 필요함.

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

multicampus

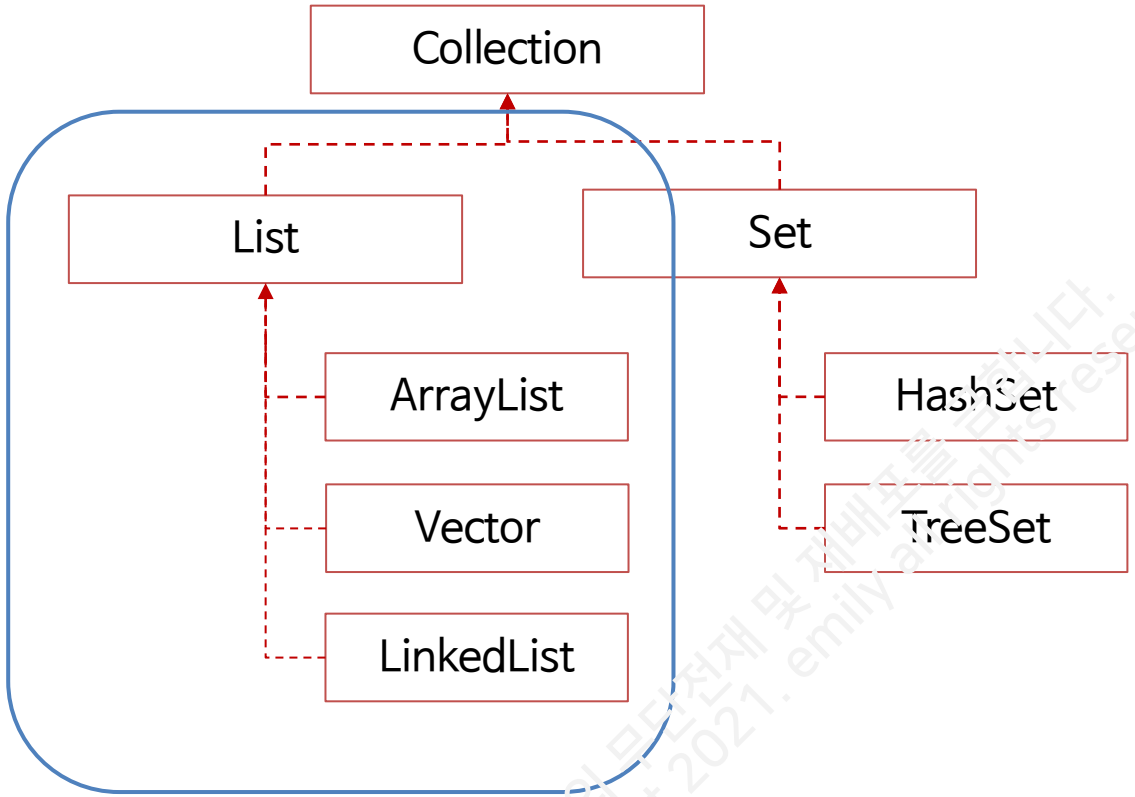
리스트

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

리스트

- 순서를 가진 데이터의 집합을 가리키는 추상 자료형
- 같은 데이터를 가지고 있어도 상관 없음

- 구현 방법에 따른 구분
  - 순차 리스트 – 배열을 기반으로 한 리스트
  - 연결 리스트 – 메모리의 동적할당을 기반으로 한 리스트



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 리스트(ArrayList)

- **구현 방법**
  - 1차원 배열에 항목들을 순서대로 저장
  - 데이터의 종류와 구조에 따라 구조화된 자료구조를 만들어 배열로 만들 수 있음

[0]	[1]	[2]	[3]	[4]	[5]
66	**33**	**44**	**55**	**99**	**77**

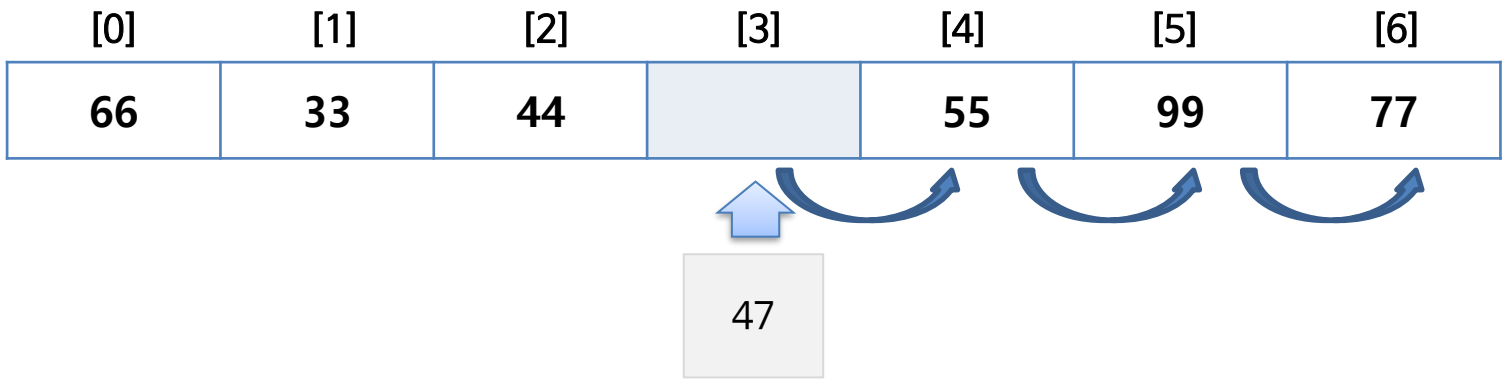
  - 데이터 접근
    - 배열의 인덱스를 이용

```
int num = arr [5]          //num=77
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 리스트(ArrayList)

- **구현 방법**
  - 삽입 연산 – 삽입 될 자료 위치를 기준으로 기존의 데이터들을 뒤로 이동하는 연산을 수행
  - 예) 47을 인덱스 3에 삽입



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 리스트(ArrayList)

구현 방법

- 삭제 연산 – 삭제 될 자료 위치의 데이터를 삭제 후 이후 데이터들을 이동하는 연산을 수행
- 예) 44를 인덱스 2에서 삭제

[0]	[1]	[2]	[3]	[4]	[5]	[6]
66	33		47	55	99	77

44

[0]	[1]	[2]	[3]	[4]	[5]	[6]
66	33	47	55	99	77	

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

순차 리스트(ArrayList)

- 배열을 이용해 순차리스트를 구현하여 사용하는 경우,
  자료를 삭제하고 삽입하는 과정에서 메모리 배열을 연속적으로 유지하기 위해 이동시키는 작업이 필요

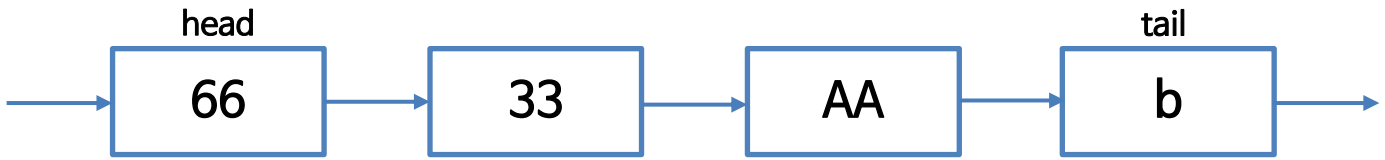
- 원소의 개수가 많고, 삽입/삭제 연산이 빈번하게 일어나면 일어날수록 작업에 소요되는 시간은 크게 증가

- 배열의 크기가 정해져 있는 경우, 삽입 시 사이즈를 늘려주는 연산이 추가되어야 하고, 삭제 시에는 순차적인 인덱스 구조로 인해 삭제된 빈 인덱스를 채워야 하기 때문에 채워주는 연산이 추가 되어야 함
  이런 부가적인 연산은 시스템 성능 저하로 이어져 삽입/삭제가 빈번하게 일어나는 프로세스에는 치명적 임
  그리고 자료들이 지속적으로 삭제되는 과정에서 그 공간만큼 낭비되는 메모리도 많아지게 됨

본 교안의 무단 배포를 금합니다.
copyright 2024 multicampus all rights reserved.
20240919

연결 리스트(Linked List)

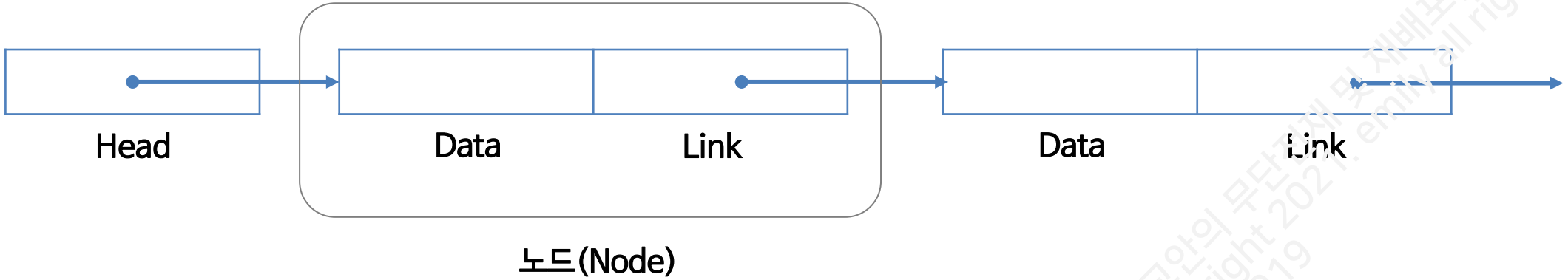
- **특성**
  - 자료의 논리적인 순서와 메모리 상의 물리적인 순서가 일치하지 않고, 개별적으로 위치하고 있는 원소의 주소를 연결하여 하나의 전체적인 자료구조를 이룸
  - 링크를 통해 원소에 접근하므로, 순차 리스트에서 처럼 물리적인 순서를 맞추기 위한 작업이 필요하지 않음
  - 자료구조의 크기를 동적으로 조정할 수 있어, 메모리의 효율적인 사용이 가능



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

연결 리스트(Linked List)

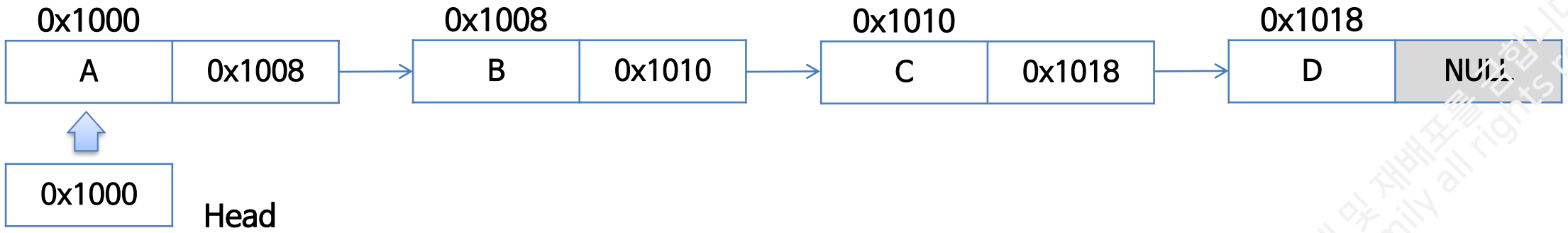
- **노드 – 연결 리스트에서 하나의 원소에 필요한 데이터를 갖고 있는 자료 단위**
  - 구성
    - 데이터 필드
      1) 원소의 값을 저장하는 자료구조
      2) 저장할 원소의 종류나 크기에 따라 구조를 정의하여 사용
    - 링크필드 – 다음 노드의 주소를 저장하는 자료구조
- **헤드 – 리스트의 처음 노드를 가리키는 레퍼런스**



본 교안의 무단 배포 및 재배포를 금합니다.
copyright 2021 emily all rights reserved.
20240919

단순 연결 리스트(Singly Linked List)

- **연결 구조**
  - 노드가 하나의 링크 필드에 의해 다음 노드와 연결 됨
  - 헤드가 가장 앞의 노드이고, 링크 필드가 연속적으로 다음 노드를 가리킴
  - 최종적으로 NULL을 가리키는 노드가 리스트의 가장 마지막 노드

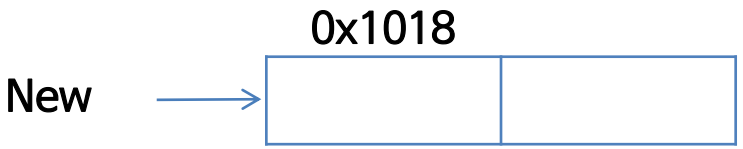


본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

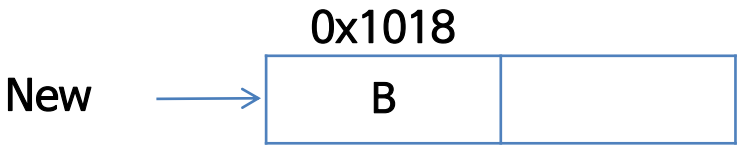
단순 연결 리스트(Singly Linked List)

▪ 노드 삽입

1) 메모리를 할당하여 새로운 노드 생성



2) 새로운 노드의 데이터 필드에 'B' 삽입

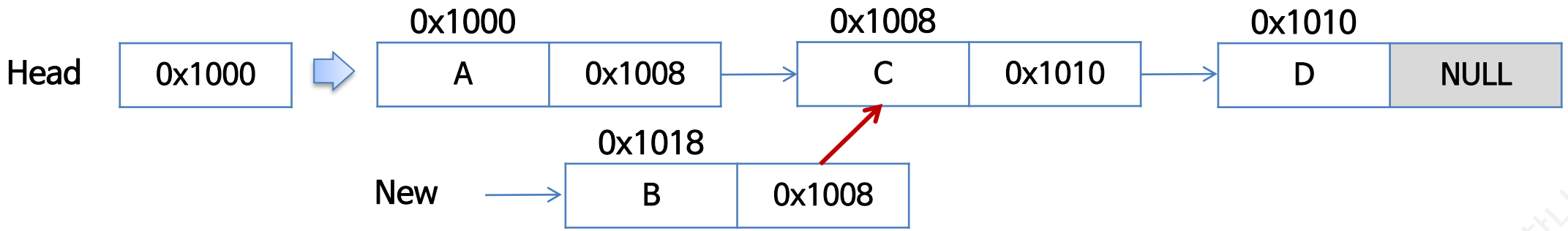


본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

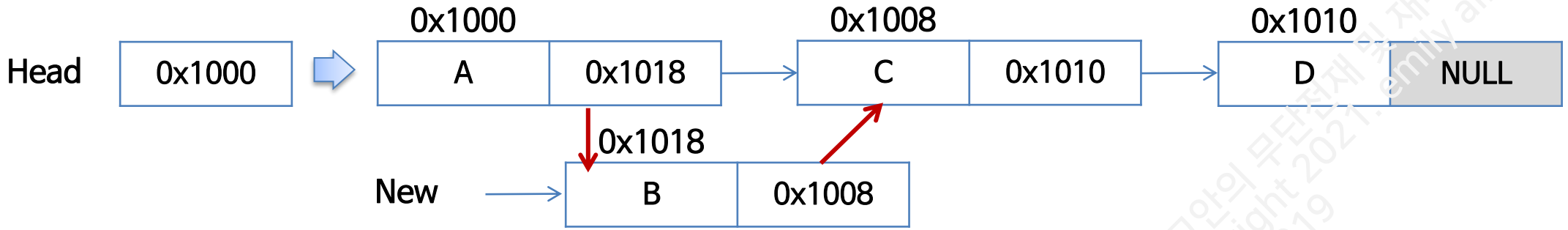
단순 연결 리스트(Singly Linked List)

▪ 노드 삽입

3) 삽입 될 위치 바로 앞에 위치한 노드의 링크 필드를 new에 복사



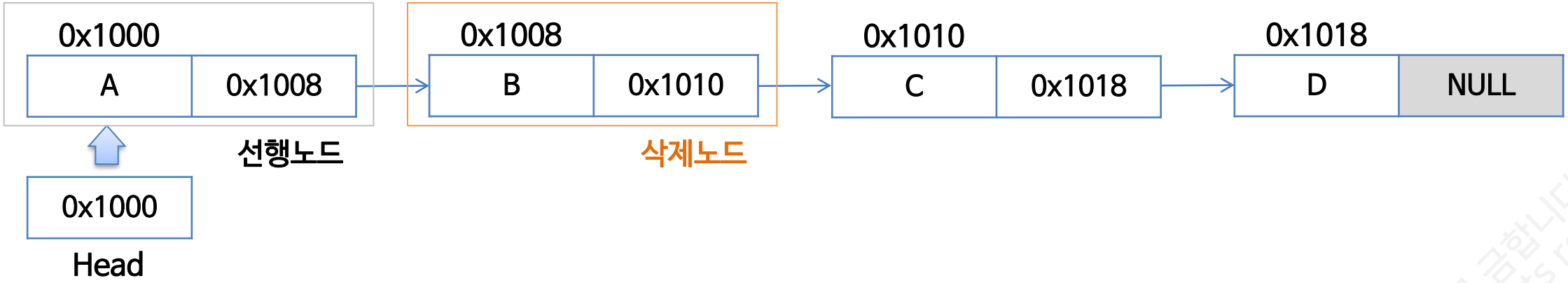
4) new의 주소를 앞 노드 링크 필드에 저장



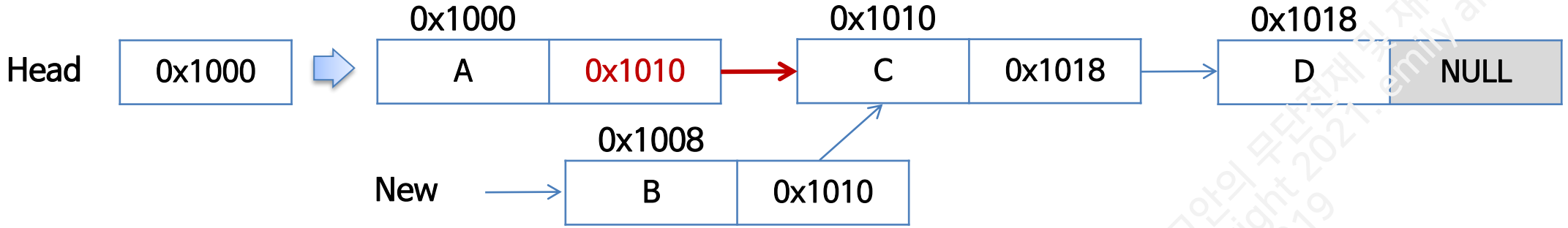
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

단순 연결 리스트(Singly Linked List)

- 노드 삭제 - 'B' 노드 삭제
  - 삭제할 노드의 앞 노드(선행노드) 탐색



  - 삭제할 노드의 링크필드를 선행노드의 링크필드에 복사

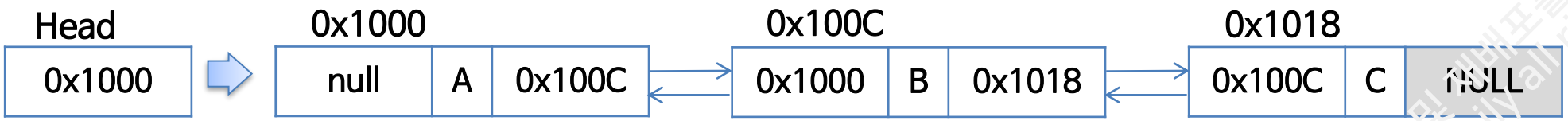


본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

이중 연결 리스트(Doubly Linked List)

- 양쪽 방향으로 순회할 수 있도록 노드를 연결한 리스트
- 두 개의 링크 필드와 한 개의 데이터 필드로 구성

- 연결 구조

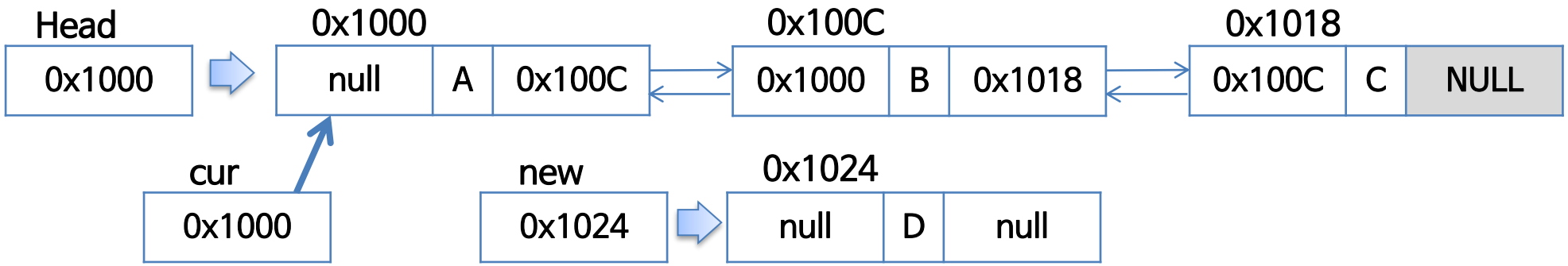


본 교안의 무단전재 및 배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

이중 연결 리스트(Doubly Linked List)

▪ 노드 삽입

1) 메모리를 할당하여 새로운 노드 new를 생성하고 데이터 필드에 'D ' 를 저장



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

이중 연결 리스트(Doubly Linked List)

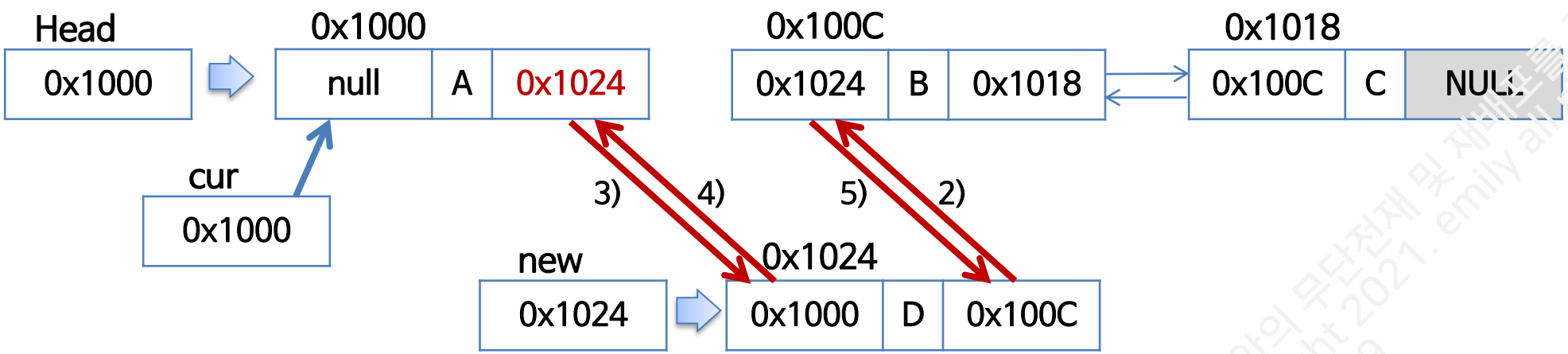
▪ 노드 삽입

2) cur의 next를 new의 next에 저장하여 cur의 오른쪽 노드를 삽입할 노드 new의 오른쪽 노드로 연결

3) new의 주소를 cur의 next에 저장하여 노드 new를 cur의 오른쪽 노드로 연결

4) cur에 있는 링크 값을 new의 prev에 저장하여 cur를 new의 왼쪽 노드로 연결

5) new의 주소를 new의 오른쪽 노드의 prev에 저장하여 노드 new의 오른쪽 노드의 왼쪽노드로 new를 연결

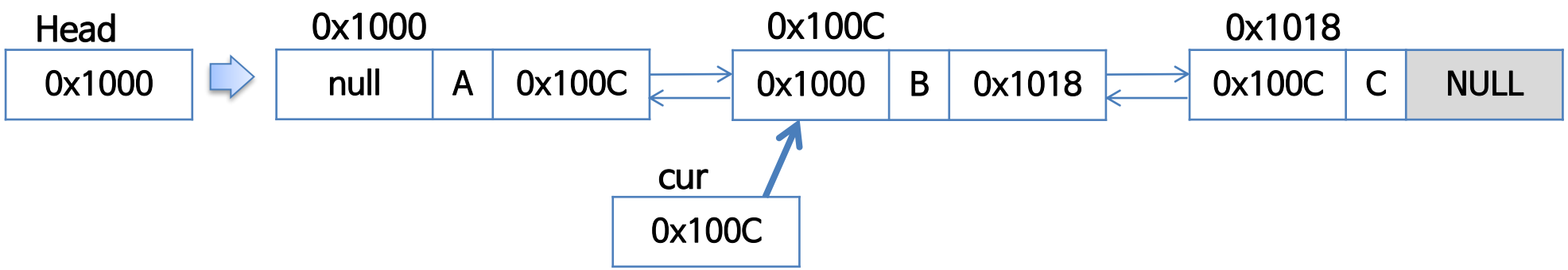


본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

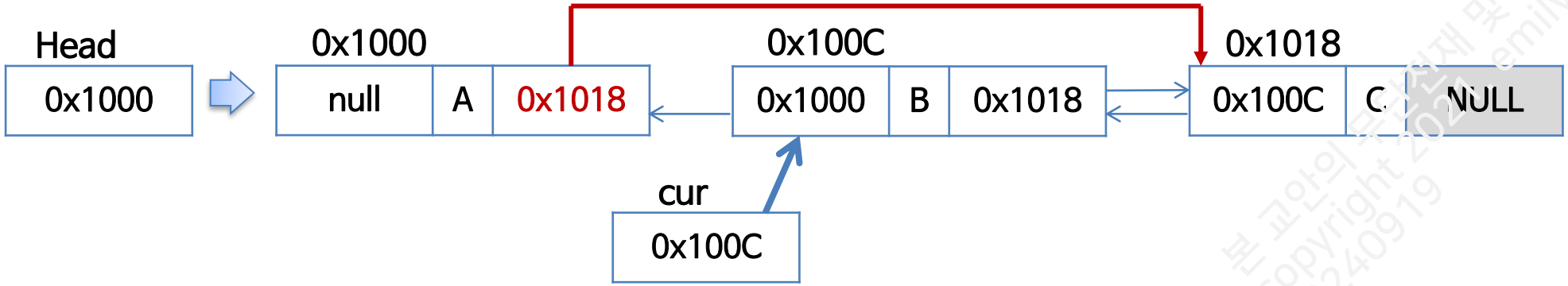
이중 연결 리스트(Doubly Linked List)

▪ 노드 삭제

1) 현재



2) 삭제할 노드 cur의 오른쪽 노드의 주소를 cur의 왼쪽 노드의 next 에 저장하여 cur의 오른쪽 노드를 cur의 왼쪽 노드의 오른쪽 노드로 연결



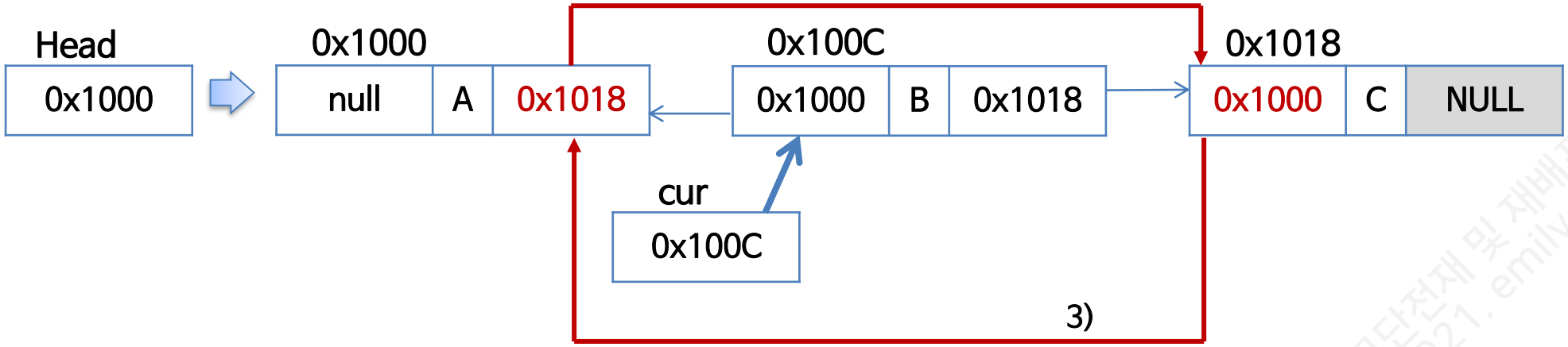
본 교안의 무단 생성 및 재배포를 금합니다.
copyright 2021 emily all rights reserved.
20240919

이중 연결 리스트(Doubly Linked List)

- **노드 삭제**

3) 삭제할 노드의 cur의 왼쪽노드의 주소를 cur의 오른쪽 노드의 prev에 저장하여 cur의 왼쪽노드를 cur의 오른쪽 노드의 왼쪽 노드로 연결

4) cur가 가리키는 노드에 할당된 메모리를 반환



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

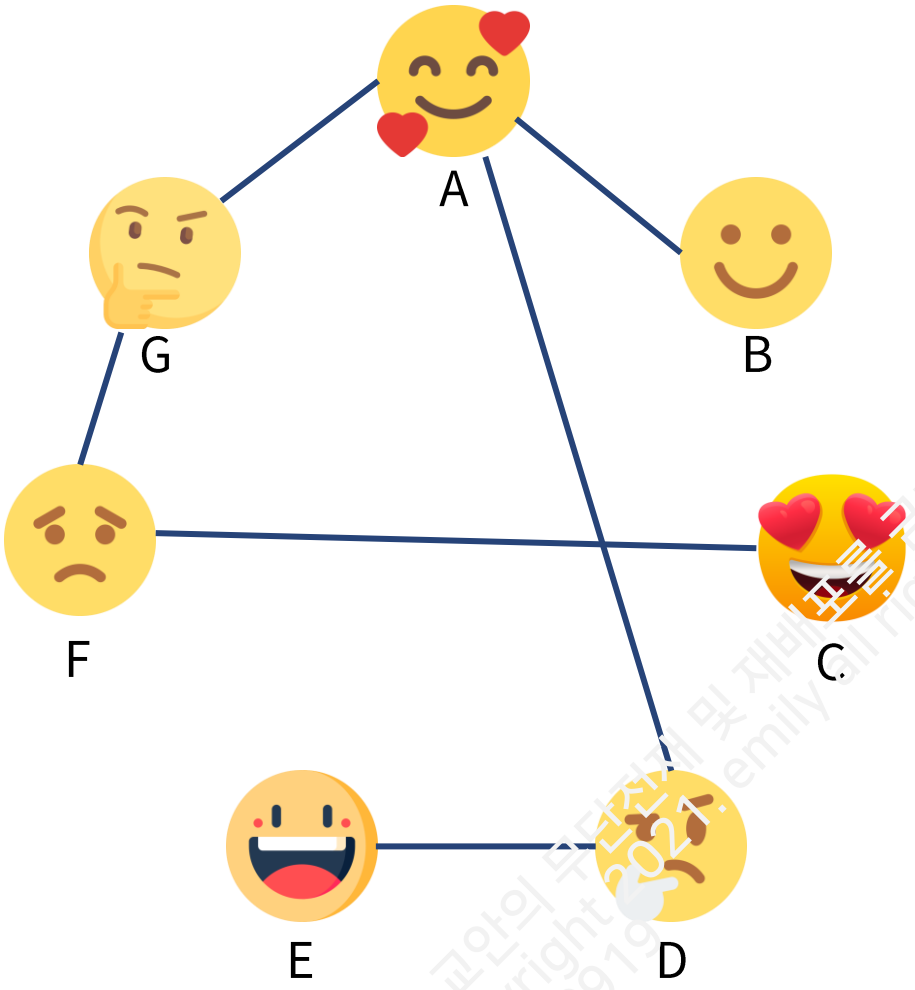
multicampus

그 래 프

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved
20240919

그래프

- 들어가기 전에…
  - A와 B는 친구
  - D와 E는 친구
  - A와 D는 친구
  - G와 F는 친구
  - A와 G는 친구
  - C와 F는 친구
  - 친구가 제일 많은 사람은?



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2024. emily all rights reserved.
20240916

그래프 알고리즘

- **객체 간의 관계를 모델링하고 분석하기 위한 중요한 도구 중 하나**
- **기본 개념**
  - 그래프는 정점과 간선으로 구성 됨. 정점은 객체를 나타내며, 간선은 정점 간의 관계를 나타 냄.
  - 간선에 방향이 있는 방향 그래프와 방향이 없는 무방향 그래프로 나뉨.
- 종류
  - 특별한 형태의 그래프로 트리(Tree) - 사이클이 없는 연결 그래프 - 와 이진 트리(Binary Tree) - 각 정점이 최대 두 개의 자식 정점을 가지는 - 가 있음.
- **그래프 탐색 알고리즘**
  - 주요 알고리즘으로 깊이 우선 탐색(DFS, Depth-First Search)과 너비 우선 탐색(BFS, Breadth-First Search)이 있음. 이 알고리즘들은 그래프의 정점을 방문하고 연결된 정점을 탐색하는데 사용
- **최단 경로 알고리즘**
  - 두 정점 사이의 가장 짧은 경로를 찾는 데 사용. (예 : 다익스트라(Dijkstra) 알고리즘, 벨만-포드(Bellman-Ford) 알고리즘)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프 알고리즘

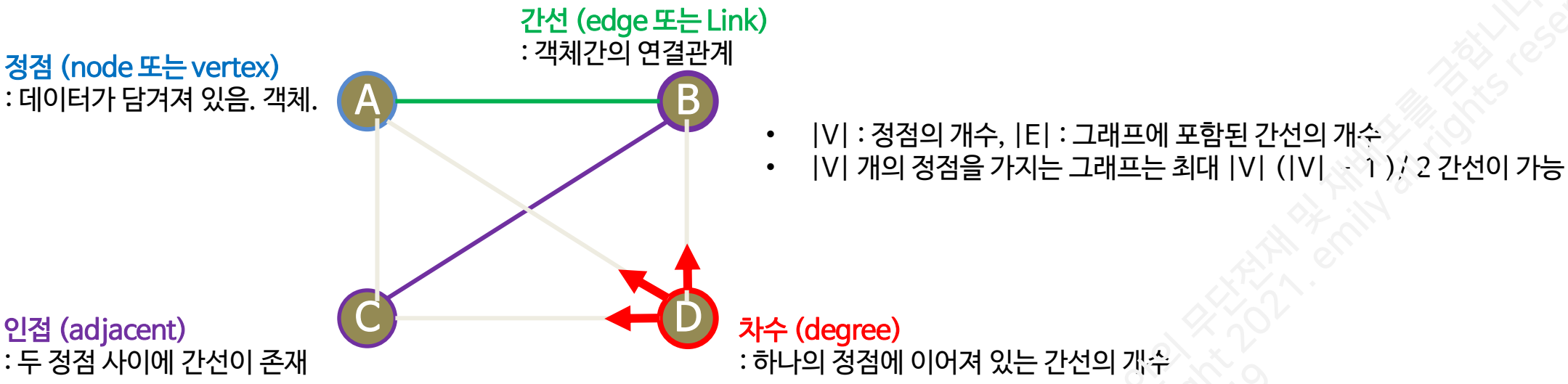
- **최소 신장 트리(Minimum Spanning Tree, MST)**
  - 그래프의 모든 정점을 포함하면서 간선의 가중치 합이 최소인 트리(예 : 프림(Prim) 알고리즘, 크루스칼(Kruskal) 알고리즘)
- **그래프 알고리즘의 응용**
  - 네트워크 최적화, 경로 찾기, 노드 간의 관계 분석, 지도 알고리즘, 소셜 네트워크 분석 등 다양한 분야에서 응용
- **그래프 라이브러리**
  - Java에서는 JGraphT, Python에서는 NetworkX 등이 사용
- **복잡성**
  - 그래프의 크기와 형태에 따라 다름.

    일부 문제는 다항 시간에 해결 가능하며(다항 시간 알고리즘), 다른 문제는 지수 시간이 걸릴 수도 있음

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프의 기본개념

- 다양한 형태의 연결 구조를 나타내는 수학적인 개념. 다른 객체들과의 관계를 나타냄.
- 정점(Vertex)의 집합과 정점 사이 연결 관계를 나타내는 간선(Edge)로 이루어진 자료구조
- 그래프는 객체들과의 관계들을 표현하기 위해 주로 사용된다
- 간선에는 방향이 있을 수 있고(A -> B) 없을 수 있다(A <-> B)
  - 다양한 방식으로 그래프를 나타낼 수 있다

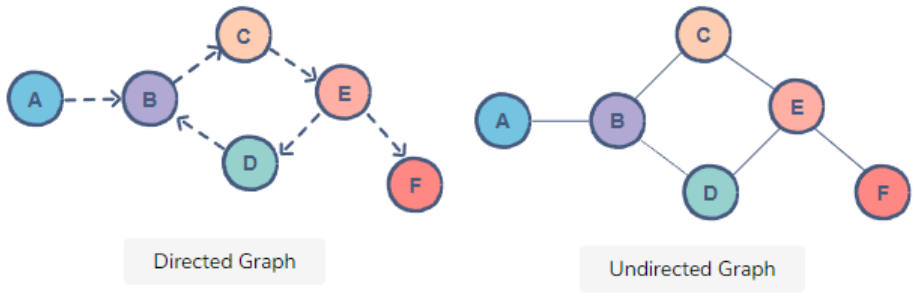


- |V| : 정점의 개수, |E| : 그래프에 포함된 간선의 개수
- |V| 개의 정점을 가지는 그래프는 최대 |V| (|V| - 1) / 2 간선이 가능

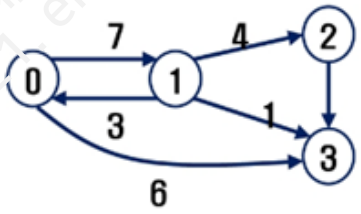
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프 종류

- **무방향 그래프(Undirected Graph)** : 간선에 방향이 없는 그래프. 간선은 양방향으로 연결.
- **방향 그래프(Directed Graph)** : 간선에 방향이 있는 그래프. 간선은 단방향으로 연결.



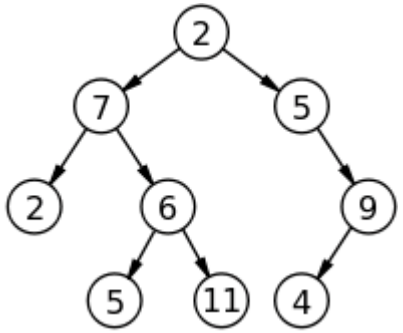
- **가중 그래프(Weighted Graph)** : 간선에 가중치(weight)가 할당된 그래프. 간선에 숫자(가중치)가 부여 됨.
  이 숫자는 두 정점 사이의 거리 혹은 비용 등을 나타냄.
- **무향 가중 그래프 (Undirected Weighted Graph)** : 무방향 그래프와 같이 간선에 방향이 없지만,
  간선에 가중치가 할당된 그래프
- **방향 가중 그래프 (Directed Weighted Graph)**: 방향 그래프와 같이 간선에 방향이 있으며,
  간선에 가중치가 할당된 그래프



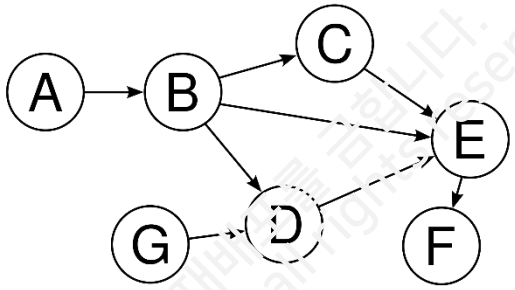
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021 emily all rights reserved.
20240919

그래프 종류

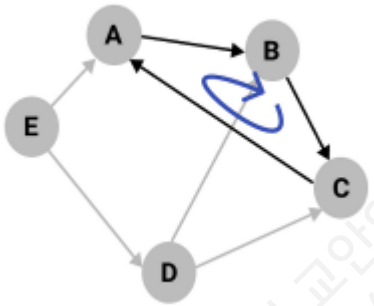
- **트리 (Tree)**: 사이클이 없는 연결 그래프.
  모든 정점이 서로 연결되어 있으며, 정점이 n개인 트리는 항상 n-1개의 간선을 가짐



- **방향 비순환 그래프(DAG, Directed Acyclic Graph)** : 사이클이 없는 방향 그래프.
  정점들 간에 방향성이 있으나 순환 구조를 가지지 않음. 위상 정렬 같은 알고리즘에 사용됨



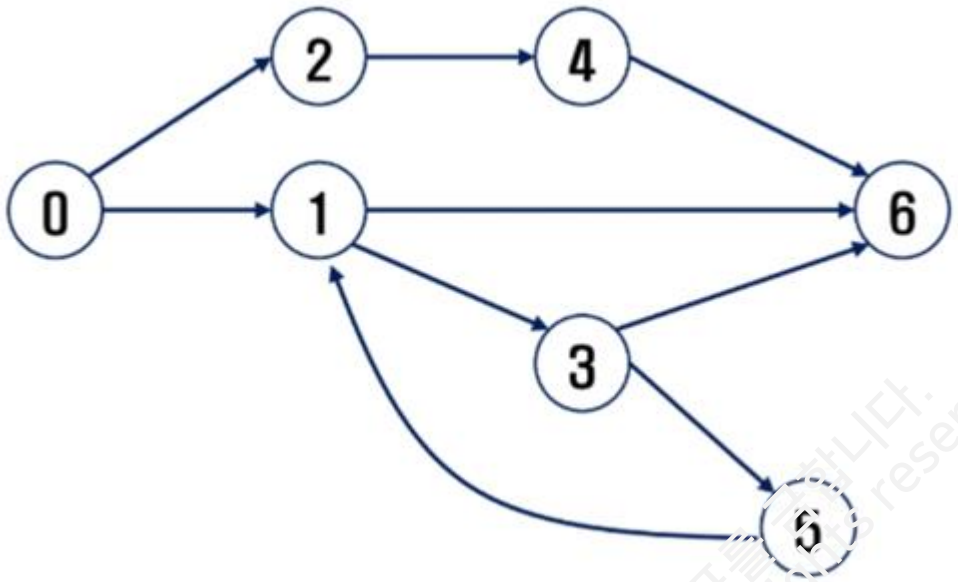
- **순환 그래프(Cycle Graph)** : 한 정점에서 시작해서 돌아와서
  다시 그 정점으로 돌아가는 경로를 가진 그래프.
  방향 그래프와 무방향 그래프 모두에서 발생할 수 있음.



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프 표현

- 인접(Adjacency)
  - 두 개의 정점에 간선이 존재(연결됨)하면 서로 인접해 있다고 함
  - 완전 그래프에 속한 임의의 두 정점들은 모두 인접해 있다
- 경로(Path) - 간선들을 순서대로 나열한 것
  - 간선들 : (0,2) (2,4) (4,6)
  - 정점들 : 0 - 2 - 4 - 6
- 경로 중 한 정점을 최대한 한 번만 지나는 경로를 단순경로라 함
  - 0 - 2 - 4 - 6, 0 - 1 - 6
- 시작한 정점에서 끝나는 경로를 사이클(Cycle)이라 함
  - 1 - 3 - 5 - 1



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프 표현

- 간선의 정보를 저장하는 방식, 메모리나 성능을 고려해서 결정한다

- 인접 행렬(Adjacent matrix)
  - |V| x |V| 크기의 2차원 배열을 활용(V : 정점의 개수)
  - 배열의 배열
- 인접 리스트(Adjacent List)
  - 각 정점마다 해당 정점으로 나가는 간선의 정보를 리스트를 활용해 저장
- 간선 배열(Edge Array)
  - 간선(시작 정점, 끝 정점)을 배열에 연속적으로 저장

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

그래프 표현

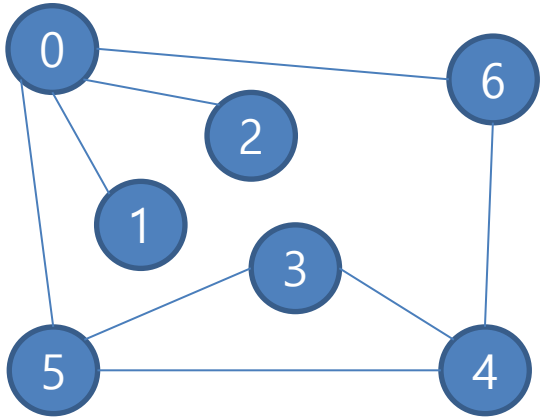
- **인접 행렬(Adjacent matrix)**
  - |V| x |V| 행렬
  - 행 번호와 열 번호는 그래프의 정점에 대응
  - 두 정점이 인접해 있으면 1, 그렇지 않으면 0으로 표현

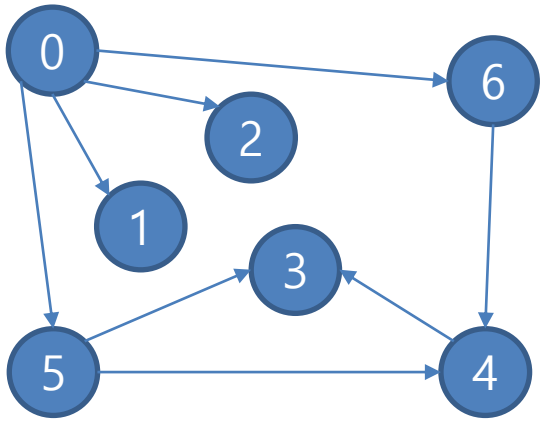
- **무향 그래프**
  - i번째 행의 합 = i번째 열의 합 = V_i 의 차수
- **유향 그래프**
  - 행 i의 합 = V_i 의 진출 차수
  - 열 i의 합 = V_i 의 진입 차수

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

인접 행렬

- 인접 행렬(Adjacent matrix)





	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1	1	0	0	0	0	0	0
2	1	0	0	0	0	0	0
3	0	0	0	0	1	1	0
4	0	0	0	1	0	1	1
5	1	0	0	1	1	0	0
6	1	0	0	0	1	0	0

	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	1	0	0	0
5	0	0	0	1	1	0	0
6	0	0	0	0	1	0	0

본 교안은 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

인접 행렬

- **장점**
  - 구현이 쉽다
  - 노드 i와 노드 j가 연결되어 있는지 확인하고 싶을 때, adjArr [i] [j] 가 1인지 0인지만 확인하면 되기 때문에 빠르게 확인 가능

- **단점**
  - 노드에 비해 간선이 적을 경우 적은 간선을 찾기 위해 전체 모두를 탐색해야 한다
  - 노드의 개수가 많고 간선의 개수가 상대적으로 적은 경우 공간 낭비가 심해 불필요한 메모리를 많이 사용하게 된다

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

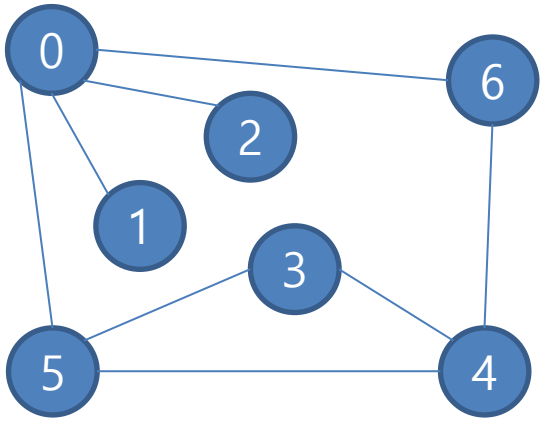
인접 리스트

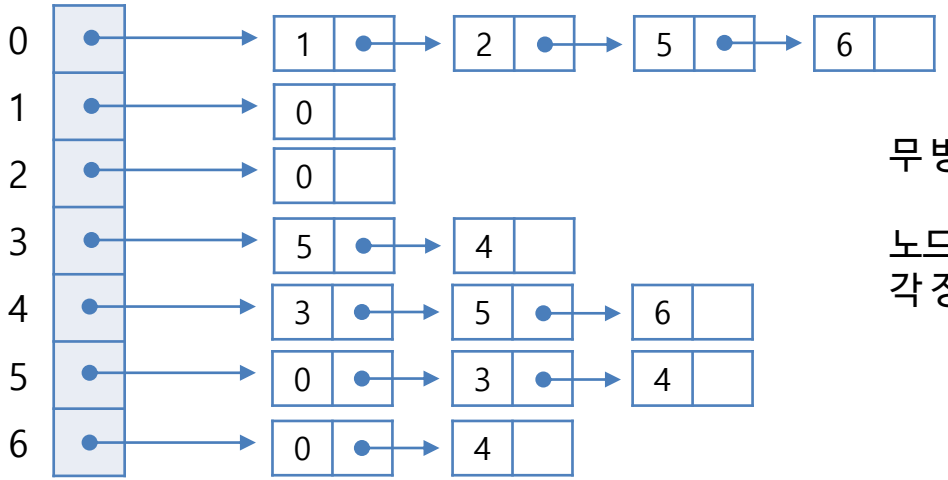
- 인접 리스트(Adjacent List)
  - 각 정점에 대한 인접 정점들을 순차적으로 표현
  - 하나의 정점에 대한 인접 정점들을 각각 노드로 하는 연결 리스트로 저장

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

인접 리스트

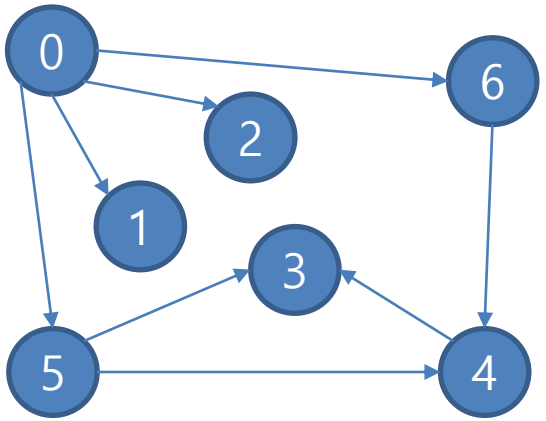
- 인접 리스트(Adjacent List)

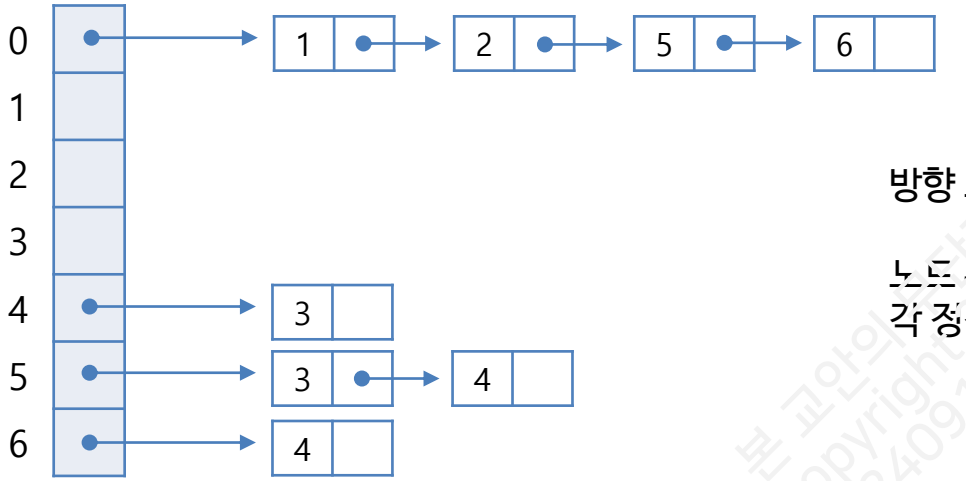




무 방향 그래프

노드 수 = 간선 수 * 2
각 정점의 노드 수 = 정점의 차수





방향 그래프

노드 수 = 간선 수
각 정점의 노드 수 = 정점의 진출 차수

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

인접 리스트

- **장점**
  - 실제로 연결된 노드들에 대한 정보만 저장
  - 모든 리스트들의 원소의 개수의 합이 간선의 개수와 같다(간선의 개수에 비례하는 메모리만 차지)

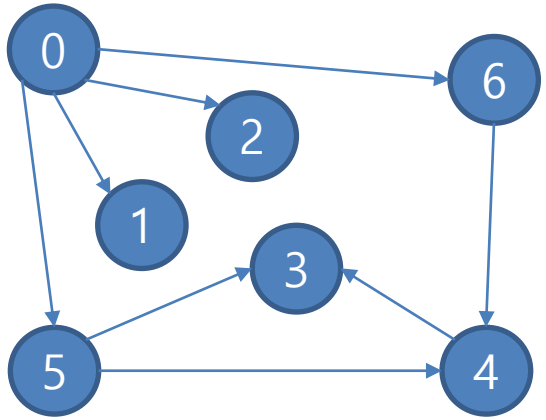
- **단점**
  - 노드 사이 연결 여부를 확인하는데에 인접 행렬보다 시간이 더 걸린다.
  - 모든 노드를 순회하고 각 노드의 인접한 노드들을 순회해야 하므로 인접 행렬보다 시간이 더 소요된다.
  - 인접 행렬은 간선의 개수가 많고 밀집 그래프에 효율적이며 인접 리스트는 간선의 개수가 적거나 희소 그래프에 효율적이다

- **그래프의 크기, 간선의 개수, 연산의 종류 등을 고려하여 결정한다**

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

간선 배열

- 간선 배열(Edge Array)
  - 정점과 정점의 연결 정보인 간선을 배열에 저장
  - 간선을 표현하는 두 정점의 정보를 배열 혹은 객체로 저장할 수 있음



간선	시작 정점	끝 정점
0	0	1
1	0	2
2	0	5
3	0	6
4	4	3
5	5	3
6	5	4
7	6	4

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

탐색 기법

- **그래프, 2차원 배열 순회(탐색)는 모든 자료(노드, 정점)를 빠짐 없이 탐색하는 것**
- **방법**
  - 깊이 우선 탐색(Depth First Search, DFS)
  - 너비 우선 탐색(Breadth First Search, BFS)

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

깊이 우선 탐색(Depth First Search)

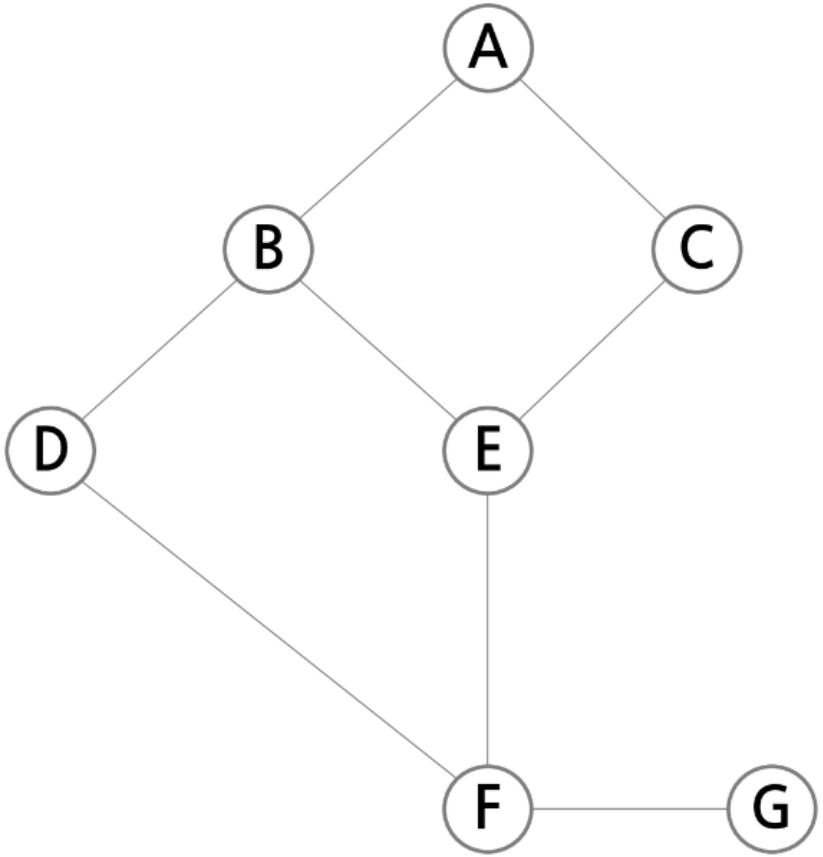
- 루트 노드(시작 정점, 출발 위치)에서 출발해 한 방향으로 갈 수 있는 곳 까지 끝까지 탐색해 더 이상 갈 곳이 없으면, 가장 마지막 다른 방향으로 갈 수 있는 노드(정점)로 되돌아와 방문하지 않은 노드(정점)로 탐색을 위와 같이 반복해, 결국 모든 노드(정점)을 방문(순회)하는 방법

- 가장 마지막에 만났던 갈림길 간선이 있는 노드로 되돌아가 다른 방향으로 깊이 우선 탐색을 반복해야 하므로, 후입선출(LIFO : Last-In First-Out)구조인 스택을 이용

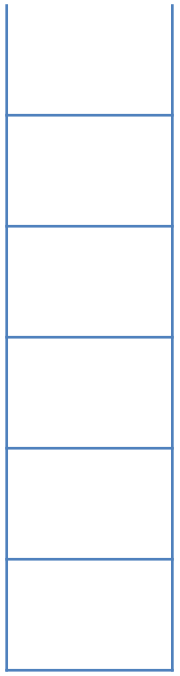
본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

깊이 우선 탐색(Depth First Search)

- 트리를 이용하여 탐색을 한다면



Stack
정점



본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

깊이 우선 탐색(Depth First Search)

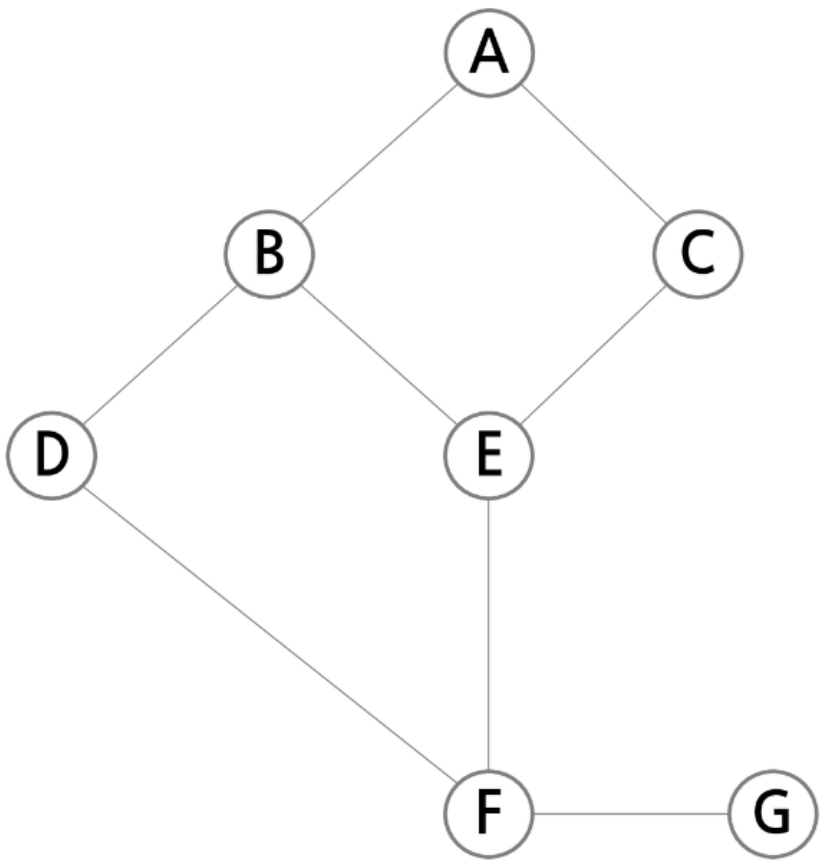
그래프 탐색 방법

```
// G: 그래프, visited : 방문배열
DFS(v) {
   visited[v] ← TRUE // v 방문설정

   FOR  each all w in adjacency(G, v)
      IF visited[w]  ≠  TRUE
         DFS(w)
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

깊이 우선 탐색(Depth First Search)

- **그래프 탐색**



정점	A [0]	B [1]	C [2]	D [3]	E [4]	F [5]	G [6]
visited	F	F	F	F	F	F	F

Stack

정점

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

너비 우선 탐색(Breadth First Search)

- 탐색 루트 노드(시작 정점)에서 자식 노드(인접한 정점)들을 모두 차례로 방문 후,
  방문했던 자식 노드들(인접한 정점)을 시작점으로 다시 해당 노드의 자식 노드(인접한 정점)들을 차례로 방문하는 방식

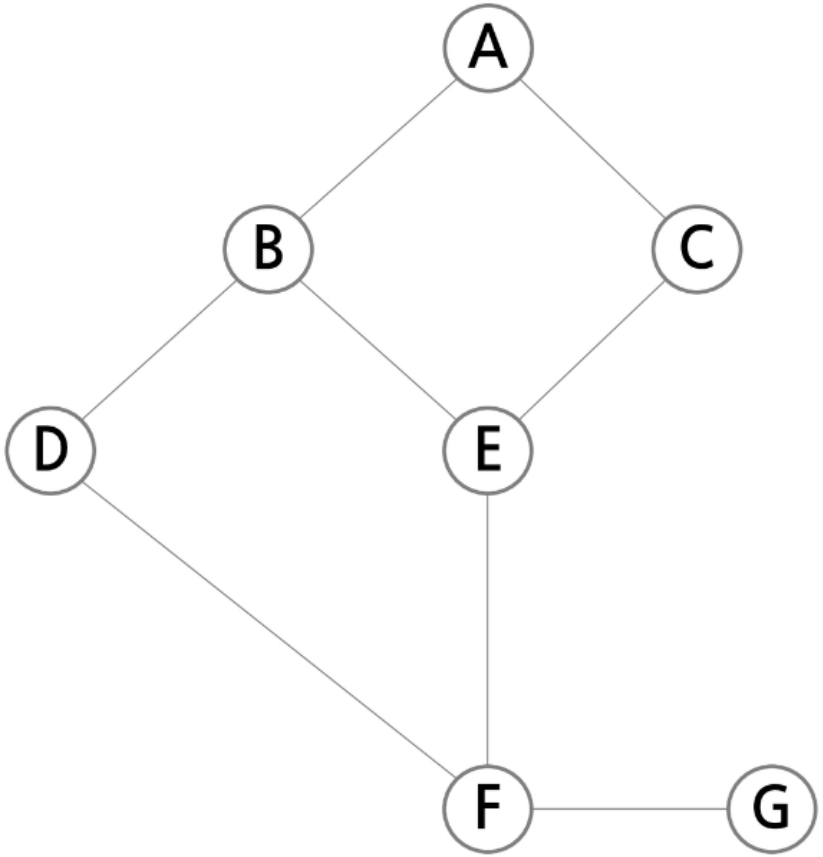
- 자식 노드(인접한 정점)들에 대해 탐색을 한 후, 다시 차례대로 너비 우선 탐색을 진행해야 하므로(갈 수 있는 정점 방문),
  선입선출(FIFO : First-In First-Out) 자료구조인 Queue를 활용

- 너비 우선 탐색은 인접한 노드들부터 차례대로 방문을 하므로 시작 정점과 끝 정점이 주어지면 최단 거리(길이)를 구할 수 있음

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

너비 우선 탐색(Breadth First Search)

- 트리를 이용하여 탐색을 한다면



Queue

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

너비 우선 탐색(Breadth First Search)

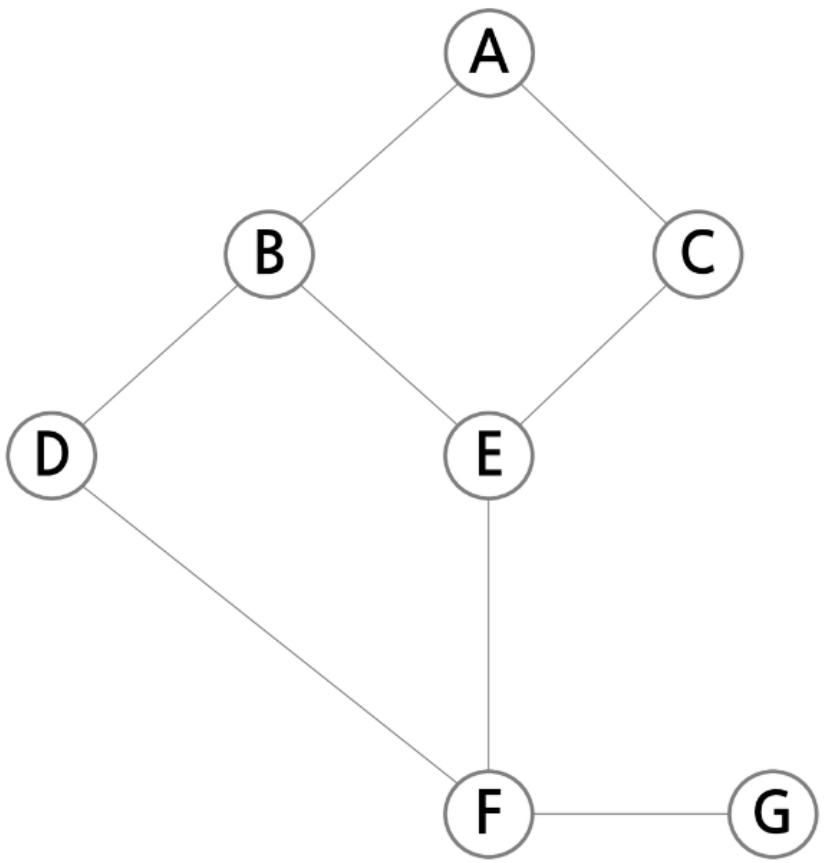
그래프 탐색 방법

```
BFS(G,v) // 그래프 G, 탐색 시작점 v
  큐 생성
  시작점 v를 큐에 삽입
  점 V를 방문한 것으로 표시
  WHILE 큐가 비어있지 않은 경우
    t ← 큐의 첫번째 원소 반환
    FOR t와 연결된 모든 선에 대해
      u ← t의 이웃점
      u가 방문 되지 않은 곳이면,
      u를 큐에 넣고, 방문한 것으로 표시
```

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

너비 우선 탐색(Breadth First Search)

- **그래프 탐색**



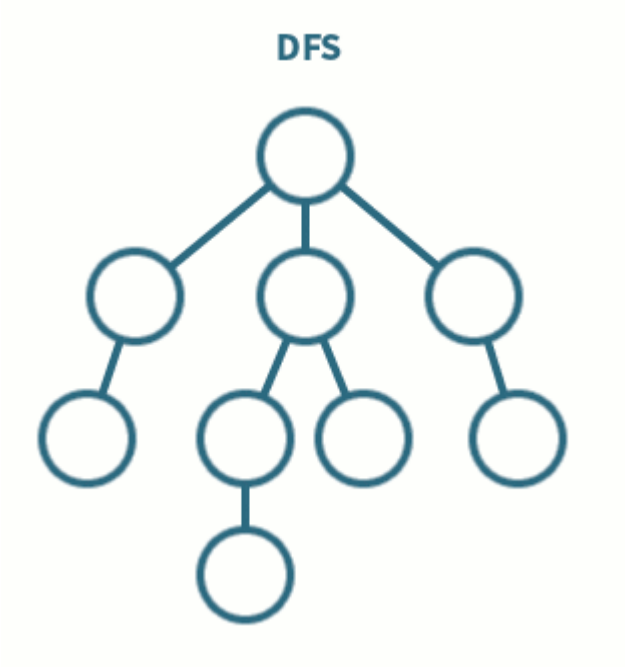
정점	A [0]	B [1]	C [2]	D [3]	E [4]	F [5]	G [6]
visited	F	F	F	F	F	F	F

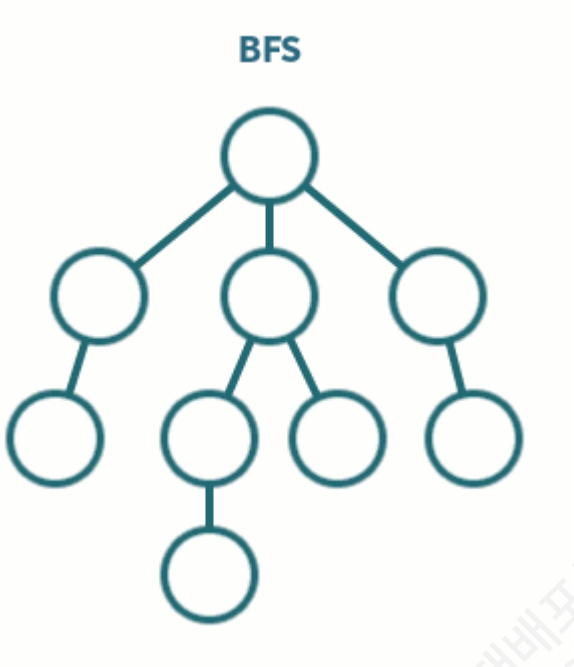
Queue

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919

DFS vs BFS

비교





장점
- 현재 경로상의 노드만 기억하면 되기 때문에 저장 공간을 많이 사용하지 않는다.

단점
- 얻어진 결과가 최단 경로라는 보장이 없다
- 무한히 깊어지면 스택오버플로우 발생할 가능성이 높다

장점
- 답이 되는 경로가 여러 개여도 최단 경로를 구할 수 있다
- 노드 수가 적고 깊이가 얕은 경우 유리하다

단점
- 큐를 이용하여 정점들을 저장하기 때문에 더 큰 저장공간이 필요할 수 있다

본 교안의 모든 내용 및 재배포를 금합니다.
copyright 2021. eouly all rights reserved.
20240919

multicampus

수고하셨습니다

본 교안의 무단전재 및 재배포를 금합니다.
copyright 2021. emily all rights reserved.
20240919